santsha2

15,286 Views

IOS-XR BGP Troubleshooting steps

Table of Contents

- Introduction
- Troubleshooting Scenarios
  - BGP session not coming UP
  - BGP session dropped/Flapped
  - BGP not advertising routes/Missing or Additional routes in RIB
  - BGP process not starting/Unresponsive/Exited
  - BGP process is consuming high CPU
  - Configuration Caveats
  - Performance and Responsiveness
  - BGP memory usage
  - BGP or BPM crash
  - Traffic loss
  - Graceful restart
  - BGP Dampening
  - BGP NSR
  - Why does one need to configure a /32 route to an ASBR peer
  - Does IOS-XR Handle slow-peer detection
  - What is IOS-XR BGP write limit
  - Multiinstance BGP

Introduction

Troubleshooting Scenarios

BGP session not coming UP

We need to check on following incase the BGP session fails to come up:

1. Check if BGP process is running
   a. Use "**`show proc bgp`**" to check the status of the BGP speaker process
   b. Use "**`show proc bpm`**" to check the status of the BPM process
2. Check if configuration is done correctly:
   a. Is AS number configured correctly
   b. the **`router-id`** is configured with a valid IP address or else a loopback address must be configured as these are the only two considered for router-id.
   c. the neighbor IP addresses are configured properly.
   d. make sure neighboring address is not configured on local box (it should be configured on the peering router).
   e. the neighbors are configured with valid **`remote-as`**.
   f. whether **`password`** needs to be configured.
   g. whether **`ttl-security`** needs to be configured.
   h. Whether **`session-mode`** configured properly on both side if configured.
   i. Check if the reason for Idle session is displayed in "`show bgp neighbor <NBR-ADDR>`" next to the BGP State. It's usually displayed inside ( ).

Use the **ping** extended command to verify that the neighbor addresses are reachable (`ping x.x.x.x size 3000 count 1000 source <src-addr>`). Also, check that for not directly connected eBGP peers, the **`ebgp-multihop`** is configured. BGP has an internal limit on the number of neighbors that can be configured. The current limit is 1024. Trying to configure the 1025th neighbor will fail and the session will not be established. It may also be useful to turn on the following debugs to find out more details:

```
a. debug bgp
b. debug bgp io [vrf <vrf>] [neighbor address]
c. debug bgp events
```

If nothing looks wrong at BGP, then look for issues in transport: Please see the attached PPT "transports_triage.ppt".

A stricter TCP connection open checks has been added in r40 as part of CSCta97299 which applies to the following scenarios:

1. IPv4 nbr with IPv6 AF:
   a. Directly-connected: Neighbor source interface must have both gbl and link-local ipv6 address
   b. Not directly-connected: Neighbor source interface (upd-src) must have global ipv6 address

2. IPv6 nbr:
   a. Neighbor source interface must have link-local ipv6 address

If the above conditions are not met then bgp will fail the open check and retry will be rescheduled. The following debug will verify the above scenarios:

```
debug bgp io <nbr>
```

A BGP session will not come up without these configurations. These requirements come from two different factors:

a. BGP needs to send both the global as well as the LL nexthops while sending update to a directly connected IPv6 neighbor and the neighbor needs both of these nexthops. Hence it is required that these addresses are configured on the interface before the IPv6 session comes up.

b. BGP needs to send a valid IPv6 nexthop with IPv6 updates being sent over an IPv4 session. The forwarding doesn't work without the IPv6 nexthop. Hence it is a requirement that the IPv6 address on the interface be configured before the IPv4 session comes up.

It is possible that IPv6 address is not avilable from ip-arm inspite of the configuration in which case collecting ip-arm traces would be a good start.

BGP session dropped/Flapped

The router console would be a good place to look for messages indicating why an existing session dropped provided **`logging console debug`** is configured. The alternative is to use the **`show logging`** command to see all the debug logs. There can be several reasons for this.

- the neighbor has gone away.
- a notification has been received from or has been sent to the neighbor identifying a problem.
- a cease notification has been received from or has been sent to the neighbor.
- there is not enough memory available on the router.

Use the **`show bgp neighbor <address>`** command to find out the status of the neighbor. This command will display the reason for the last session reset. Also, any notification data that has been sent or received before resetting the session is displayed. BGP sets an internal limit (can be configured using the `maximum-prefix configuration` command) on the number of prefixes it will accept from a neighbor for a given AFI/SAFI command. The session will be reset if the number of prefixes exceeds this limit.

if a notification has been send and received, http://wwwin-routing.cisco.com/cgi-bin/bgp_decode/bgp_decode.pl can be used to decode the notification msg.

A typical scenario is that holdtime expire because of big packet get dropped. Please try ping with big size and longer duration to see whether big packet get dropped by somewhere (`ping x.x.x.x size 3000 count 1000`).

If nsr is configured, an extended ping (big size) from both active and standby would be helpful.

Another typical scenario is that peer close down the session without notification. It could be that tcp connection get dropped. Need to follow tcp triage to see what's wrong: Please see the attached PPT "transports_triage.ppt"

.

You can get a packet trace from tcp for a session.

If the session is still up, do the following:

1. `show tcp brief | inc <neighbor address>`

   To get the pcb address

2. `show tcp pack-trace <pcb address> location ..`

   Will give you the packet trace for this current session.

If the session has already gone down, you might be able to find the packet trace as:

1. `show tcp dump-file list <nbr> location ...`

   You get a list of tcp dump files for the neighbor. Find the dump-file that corresponds to the session that went down. You should be able to tell by the timestamp. The file would be created at around the time the session went down.

2. `show tcp dump-file <file name> location ...`

   This will print the packet traces for the session. From the packet trace, you should be able to see why the pcb got closed. If not, then collect the output and send it to the BGP/TCP team.

BGP not advertising routes/Missing or Additional routes in RIB

BGP not in read-write mode

We may not see the routes in the BGP table incase the BGP has not left the read-only mode. Following commands will help to see if the BGP has left the read-only mode and entered read-write mode:

- `show bgp`
- `show bgp process [performace-statistics] detail`

The former command will show the entire BGP table. If already in read-write mode, then BGP best paths have been selected and are represented with **>**. If BGP contains a large number of routes, the show command could take a long time to complete. For this reason, the latter command is more recommended. If BGP is already in read-write mode, you will be able to see the following lines at the very end of the show command:

```
First neighbor established: <time>
Entered DO_BESTPATH mode: <time>
Entered DO_RIBUPD mode: <time>
Entered Normal mode: <time>
```

BGP routes are dropped by the Table Policy

A table policy is used to filter routes that are being installed in the RIB and/or change some of the attributes of the routes (i.e. adding traffic-index). An incorrectly configured route-policy will end up preventing routes from being installed in the RIB.

Invalid next hop

If a BGP route does not have a valid next hop, then it will not be installed in the RIB. Use show bgp <pfx>/len to see whether it has a valid next hop. In the following example, BGP route has an invalid (inaccessible) next hop.

```
  BGP routing table entry for 192.169.0.0/24
      Versions:
        Process           bRIB/RIB  SendTblVer
        Speaker               2929        2929
      Paths: (1 available, no best path)
        Not advertised to any peer
        Received by speaker 0
        Local
          12.0.200.1 (inaccessible) from 10.0.101.2 (10.0.101.2)
            Origin IGP, localpref 100, valid, confed-internal
```

In case of biscuit, you need check the following:

1. tunnel template is configured

```
RP/0/1/CPU0:router#show running-config tunnel-template
Fri Sep  5 14:03:02.176 UTC
tunnel-template test
 mtu 1410
 tos 7
 ttl 128
 encapsulation l2tp
 !
 source 22.22.22.22
!
```

2. show bgp ipv4 tunnel

```
RP/0/1/CPU0:Lumina#show bgp ipv4 tunnel
Fri Sep  5 14:01:43.369 UTC
```

```
BGP router identifier 22.22.22.22, local AS number 1
BGP generic scan interval 60 secs
Non-stop routing is enabled
BGP table state: Active
Table ID: 0xe0000000
BGP main routing table version 3
BGP NSR converge version 3
BGP NSR converged
BGP tunnel nexthop version 3
BGP scan interval 60 secs

Status codes: s suppressed, d damped, h history, * valid, > best
              i - internal, S stale
Origin codes: i - IGP, e - EGP, ? - incomplete
   Network            Next Hop            Metric LocPrf Weight Path
*> 1:22.22.22.22/48   0.0.0.0                                 0 i
*>i65535:10.0.101.10/48 10.0.101.10                   100      0 i

Processed 2 prefixes, 2 paths
```

a. make sure there are local tunnel prefix (if local tunnel prefix is not there, remote tunnel prefix might show as inaccessible) If local tunnel prefix is not there, get "`show tunnel-ip trace ma all reverse`" check what's the last cmd tunnel ma send to bgp.

b. make sure there are remote tunnel prefix (if not, check a on the the other side of the tunnel). c. make sure the prefix do have bestpath

3. `show rib opaques safi-tunnel`

Make sure that the tunnel next hop exists.

```
RP/0/1/CPU0:Lumina#show rib opaques safi-tunnel
Fri Sep  5 14:07:00.284 UTC
Summary of safi tunnel opaque data in IPv4 RIB:

Opaque key: 65535:10.0.101.10
Opaque data&colon;
Tunnel Encap - ifhandle=0x1000180, type=L2TPv3, Params=[Session-id=0x1F6B7100, b
Cookielen=8, Cookie=0x1234567887654101]
RP/0/1/CPU0:Lumina#
```

4. `show cef recursive-nexthop`

Make sure that the tunnel recursive next hop exists

```
 RP/0/1/CPU0:Lumina#show cef recursive-nexthop
Fri Sep  5 14:10:27.904 UTC

Recursive nexthop database:

10.0.101.10/32, flags 0x0
  Recursive nexthop is resolved
  Resolving route is 10.0.0.0/16
    Dependants:
    * Tunnel endpoint, tunnel id 65535, flags 0x301, ref count 5
      Local tunnel is up
      Tunnel endpoint is resolved
      Tunnel Encap - ifhandle=0x1000180, type=L2TPv3, Params=[Session-id=0x1F6B7100, Cookielen=8, Cookie=0x1234567887654101]

Summary:
            Number of recursive nexthops:      1
             Resolved recursive nexthops:      1
           Unresolved recursive nexthops:      0
          Recursive nexthops not tracked:      0

              Number of tunnel endpoints:      1
     Number of resolved tunnel endpoints:      1
   Number of unresolved tunnel endpoints:      0

RP/0/1/CPU0:Lumina#
```

BGP is stuck

It could be possible that BGP has a thread which is spinning and, therefore, depriving RIB Update thread from necessary CPU time to install routes. Run **`show process bgp`** and **`top`** to find out whether a BGP thread is hogging the CPU. If so, provide the following outputs:

```
show process bgp
top
run attach_process -p <BGP PID> -t <BGP Thread>
```

Route is being dropped by Route-policy

Check the appropriate route-policy (out-bound policy for advertisement and table policy for RIB installation) to make sure that it is not dropping the prefixes

Label Space is exhausted

It is possible that BGP has been unable to allocate label for all the VPN prefixes because the number of labels required is more than what is allowed by LSD. This will result in the prefixes without local label to be absent from the RIB. They will not be advertised to any neighbor either. BGP puts out an IOS msg indicating that there has been a label space exhaustion.

```
RP/0/0/CPU0:Aug 22 17:21:40.298 : bgp[123]: %ROUTING-BGP-3-ERR_ALM_ONE_LABEL : [3] : Unable to allocate label: 'MPLS_LSD'
detected the 'resource not available' condition 'Code(0)': No space left on device
```

This state can also be seen using the **`show bgp vpnv4 unicast label`** or the **`show bgp vpnv6 unicast label`** commands.

```
RP/0/0/CPU0:corvette#sh bgp vpnv6 u lab

Mon Aug 25 19:15:27.821 UTC
BGP router identifier 1.2.3.4, local AS number 1
BGP generic scan interval 60 secs
Non-stop routing is enabled
BGP table state: Active
Table ID: 0x0
BGP main routing table version 11
BGP NSR converge version 1
BGP NSR converged
BGP scan interval 60 secs
MPLS label space exhausted <<<<=============

Status codes: s suppressed, d damped, h history, * valid, > best
              i - internal, S stale
Origin codes: i - IGP, e - EGP, ? - incomplete
   Network            Next Hop        Rcvd Label      Local Label
Route Distinguisher: 900:1 (default for vrf 900)
*> 2222:2222::/112    9000::1001      nolabel         16015
Route Distinguisher: 901:1 (default for vrf 901)
*> 3333:2222::/112    9100::1001      nolabel         16016
Route Distinguisher: 902:1 (default for vrf 902)
*> 4444:2222::/112    9200::1001      nolabel         16017
Route Distinguisher: 903:1 (default for vrf 903)
```

```
*> 5555:2222::/112    9300::1001      nolabel         16018
Route Distinguisher: 904:1 (default for vrf 904)
*> 6666:2222::/112    9400::1001      nolabel         16019
Route Distinguisher: 905:1 (default for vrf 905)
*> 7777:2222::/112    9500::1001      nolabel         16014
```

MPLS label space can be modified using the **`mpls label range`** configuration.

BGP process not starting/Unresponsive/Exited

BGP not responding

If BGP stops responding to commands, or peer connections have been dropped, the first thing to check is whether the process is still running:

In standalone mode, use:

- **`show proc bgp location all`** to check the status of the BGP process,
- **`show proc bpm location all`** to check the status of the BPM process.

In distributed mode use:

- **`show proc bgp location all`** to check the status of the BGP speaker processes,
- **`show proc brib location all`** to check the status of the bRIB processes.
- **`show proc bpm location all`** to check the status of the BPM process.

In distributed mode there is a entry for each speaker process in the output of **`show proc bgp location all`**, and an entry for each bRIB process (IPv4 or IPv6) in the output for **`show proc brib location all`**.

The ID of the speaker process can be determined from the hexadecimal number at the end of the pathname for the Started on config field. The standalone BGP process has ID 0, and the ID for distributed speaker processes corresponds to the speaker-id.

The bRIB processes have IDs 81 (hex) for the IPv4 bRIB and 82 (hex) for the IPv6 bRIB.

Note: The output of show proc may include details of processes no longer expected to be running. Provided the status of these processes is Exited, this is not a problem.

From 4.2 onwards, the Tag in the output of **`show proc bgp location all`** corresponds to the BGP instance name.

Process not running

If the show proc output indicates that the correct processes are running, go to Process running but not responding.

If distributed BGP processes are running, and BGP is configured to be in standalone mode, or vice versa it may be that the mode change is still pending. Issue the command **`clear bgp *`** to make the mode change take effect.

In distributed mode, it should be noted that speaker processes are not started until a neighbor is assigned to that speaker. To assign a neighbor to a particular speaker

process, use the **`neighbor <addr> speaker-id <id>`** configuration command. If the neighbor is already assigned to another speaker process, it must be reset using the **`clear bgp <addr>`** command for the new assignment to take effect.

Collect the following information to help determine why the process isn't running:

- Output from **`show process bpm location all`** , **`show process bgp location all`** and **`show process brib location all`**
- Output from **`show process log`**
- Output from **`show logging`**
- Information about processes BPM has attempted to start:
- Type **`run sysdbcon`**
- Enter **`ipc/gl/ip-bgp/`** as the initial bind point
- Enter **`iterate`**
- Enter **`quit`**
- Check whether there have been any BGP processes crashes. Check output of **`show context all location all`**. If there are crashed processes, collect the following:
- Output from **`show context all location all`**
- Core files referenced in show context output.
- Output from **`show version`**
- Attempt to restart the process using the **`proc restart <jid> location <node>`** command for each process that is expected to be running. If the process does not successfully restart, collect any syslog or console output (use **`show logging`**)

From 4.2 onwards, collect the following information to help determine why the process isn't running:

- Output from **`show process bpm location all`** and **`show process bgp location all`**
- Output from **`show logging`**
- **`show bgp instances`** to check the BGP instance names and their Placed group names
- **`show placement program bgp`** to check the spawning status of the active/standby BGP process by Placed
- **`show bgp trace location all | in Placed`** to check the status of the BGP/Placed interaction

If the spawning status of the BGP process is not **`RUNNING`**, please check further with Placed team. If you see that matching callback function to Placed API is missing in the output of **`show bgp trace location all | in Placed`**, please check further with Placed team.

Note that, in 4.2 currently, when BGP process is not running, **`proc start [bgp|<bgp jid>]`** will not start the process due to existing Placed DDTS. The workaround is to unconfigure and reconfigure of the BGP instance.

Process running but not all Threads are created

If BGP process is running, but not all threads are created, the process is waiting for the complete receipt of the initial configuration from BPM. This is highly likely AIPC transport issue. Please confirm it by collecting the following information:

- **show bgp instances** to identify internal BGP instance id
- **show bgp trace aipc location all | in bpm send msg** to identify messages sent by BPM---check with buff-seq
- **show bgp trace aipc location all | in last-batch** to confirm if the last message containing the initial configuration is sent by BPM
- **show bgp trace aipc location all | in bgp ipc rcv** to confirm if BGP process has received all the messages from BPM---check with buff-seq

If BPM sent the all the messages to BGP process, but the BGP process didn't receive all of them, please check with AIPC team for further investigation.

Process running but not responding

If the process is running, but is not responsive, there are several possibilities:

- The process is blocked, either against itself, or another process.
- The process is stuck in a tight loop
- The process/system is busy, but forward progress is being made.

If the output from **show process <procname> location all** indicates that all threads are in Receive state, the process should be responsive and operating normally.

If the thread states in the output of **show process <procname>** change over time in successive executions of the command, it is unlikely that BGP is stuck in a loop, or is blocked. Collect the output from **top dumbtty** for several iterations. If BGP processes consistently appear with high CPU usage, it is most likely that there is lots of network churn - See section on Performance and responsiveness. If the CPU usage is thought to be excessive, collect the information detailed in Process using all available CPU.

If there are threads consistently not in Receive state in the output of **show process <procname>** command, there may be a problem with the process. If the suspect threads are in Ready state, go to Process using all available CPU, otherwise it is likely that the process is blocked - collect the following information:

- Output from **show version**
- Output from **show process <jid>** (JID is the Job ID in the **show process** output).
- Output from **show process blocked location all**
- Output from **run pidin**. This command must be executed locally on each node where BGP processes are running.
- Output from **show process threadname <jid> location <node>**
- For each thread that is in Ready state, output from **run attach_process -p <pid> -t <tid> -v** (PID and TID appear in the output of **show process <procname>**). Note this command must be executed on the node on which the process is running.
- For each thread that is not in Receive or Ready states, output from **run attach_process -p <pid> -t <tid> -v -i 1 -f**. Note this command must be executed on the node on which the process is running.
- Output from **show dll jobid <jid> location <node>**
- Provided it is OK to bring BGP sessions down, collect a core file for the unresponsive process. Use the **dumpcore suspended <jid> location <node>** command.

Once the above information has been collected, it may be possible to rectify the problem by entering **proc restart <jid> location <node>** for each unresponsive process.

BGP process is consuming high CPU

Below are the most probable reason for high CPU:

1. BGP receiving continuous route updates
2. BGP table version is wrapped up

BGP receiving continuous route updates

In this case we will see BGP table versions to increase very rapidly. Collect followin logs 10 times at 30 sec interval to get the churn rate:

- `show bgp all all summary | i main`
- `show bgp <afi> <safi> summary | i main` (incase afi/safi is known)

For normal internet churn, ~100 version/min is expected and should not cause high CPU. However, further monitoring is required incase the version increases at > 1000/min.

One of the common cases where BGP uses high CPU is due to continuous VPNv4/6 table leading to continuous label and import activity. To identify this we need to get the thread information and montior the CPU usage by the threads.

Following steps can be used to get the required info:

- get the job id of bgp process using "`show process bgp`"
- Get the thread names using "`show process threadname <bgp_jid>`"
- Get the CPU usage by various threads using "`show process <bgp_jid>`"

```
show process threadname <bgp_jid>
1045   3    bgp-label       10 Receive        0:00:00:0117 bgp
1045   4    bgp-rib-upd-0   10 Receive        0:00:00:0061 bgp
1045   8    bgp-import      10 Receive        0:00:00:0068 bgp
1045   9    bgp-upd-gen     10 Receive        0:00:00:0042 bgp
```

```
show process <bgp_id>
1045     8  10 Rcv   106:29:30    39.62% bgp (import thread)
1045     9  10 Rcv    22:02:25     8.33% bgp (update-gen thread)
```

```
1045     4  10 Rcv    21:18:43     7.77% bgp (rib-update thread)
1045     3  10 Rcv    13:42:40     5.19% bgp (label thread)
```

This indicates that BGP is receiving frequent VPN updates for which BGP is doing import, update generation, and installing the imported routes into RIB. The heavy CPU usage by the import thread is caused by the fact that pre-4.1.0, BGP walk all the RDs during every import walk. So, even if we receive one VPN prefix per sec, BGP will end up walking all the RDs. This behavior has been changed in 4.1.0 where BGP does a versioned walk of the RDs as a result only walks the RDs for which updates have been received and not all the RDs.

following knobs can be used to control the CPU usage:

- `bgp import-delay <sec> <msec>`
- `bgp label-delay <sec> <msec>`

These knobs batch the import and label walks, so that instead of walking the RD as soon as a new update/withdraw is received, the walks are done at certain interval as specified by the knobs.

This needs to be specified under the VPNv4 address family. The higher the delay lower will be the CPU usage. Please note that the delay configured will have impact on convergence. BGP will delay the import/label walk, which in turn will delay the BGP update advertisements.

With a delay of 1sec, the worst delay will be 1 sec. Consider the case when a node receives the routes at t=0, t= 100ms, t=500ms and t= 900 ms. In this the delays encountered by each of the prefixes will be 1 sec, 900 ms, 500 ms and 100 ms respectively.

BGP table version is wrapped up

Each BGP address-family has an independent 32 bit version number that is incremented whenever a prefix in that table needs processing. Once the table version reaches the max value of 2^32 -1, it wraps and starts over from 1. At that point we lose the scoped table walk optimisation. Each tree operation may end up traversing the entire prefix table and hence result into significant delay.

How to identify if BGP table version is wrapped up:

1. Check if RD version is greater than the BGP table version. following CLIs will help to check on this:

```
show bgp <afi> <safi> summary
show bgp vrf all summary | inc "VRF:|RD|main"
```

2. Check if the BGP table version is smaller than any prefix version in that table.

3. Check "`show bgp trace`" to see if version wrap trace is present:

```
Mar 10 03:43:47.777 default-bgp/spkr-tr2-ev 0/4/CPU0 t16 [EV]:147: Ver wrapped for AF 'VPNv4 Unicast'
```

4. Provide bgp core file and version info (shown below) to BGP team to find out internal state to confirm if version wrap has occurred in past or not.

```
(config)# exception sparse off
(config)# exception coresize 4095
(config)# commit
dumpcore running bgp
show install which comp ipv4-bgp
```

CSCun63547 is tracking the enhancement where we retain the scoped walk optimization even after the version wrap. Once the fix is present, the CPU utilization should remain the same, provided the system dynamics don't change. The fix needs changes in the internal radix tree library that BGP along with several other components use.

Please collect following following logs for further debugging of issues related to high CPU:

- `show bgp all all process performance detail (gets the BGP scale)`
- `show install which comp ipv4-bgp`
- `top d for a period of 5 min`
- `show process <bgp_jid> location <node> (JID is the Job ID in the show process bgp output)`
- `show process threadname <bgp_jid> location <node> (<node> is the RP on which the active BGP process is running)`
- `run attach_process -p <pid> -i 10 (PID is found inshow process <procname>). Note this command must be executed on the node on which the process is running.`
- `follow job <bgp_jid> stackonly iteration 20 delay 1`
- `show dll jobid <bgp_jid> location <node>`
- `show tech routing bgp`
- `dumpcore running <bgp_jid> location <node> command.`

Configuration Caveats

There are some un-intuitive configuration behavior in BGP, which stems from the two-stage configuration architecture in IOX-XR, and the fact that configuration changes are batched for performance reasons. In combination, these factors mean that configuration is not always verified and applied in the same order that the user entered it, or the same order that they see it when they do **show run**, and this can lead to some unexpected behavior.

There are only three cases where the caveats manifest themselves and the proposed solutions are outlined below

remote-as

The fact that a neighbor must always have a remote-as defined means you can run into problems when inheriting remote-as from a group, for example:

Starting config:

```
router bgp 1
  neighbor-group foo
    remote-as 1
  exit
  neighbor 1.2.3.4
    use neighbor-group foo
    update-source loopback0
  exit
exit
```

Change:

```
router bgp 1
  neighbor 1.2.3.4
    remote-as 1
    no use neighbor-group foo
  exit
exit
```

This fails since the **no use** command is processed first, and that leaves the neighbor with no remote-as and hence the command is rejected. The following restrictions must be observed to avoid such a situation.

Restriction: The following commands should not be combined in the same commit within the IOX-XR configuration architecture.

- **remote-as** commands and **no use neighbor-group** commands
- **remote-as** commands and **no use session-group** commands

Address families

A similar situation occurs with address-families, which must be activated before any other configuration is allowed. The analogous problem to the remote-as case can occur when the activation is inherited. However, the structure of the CLI prevents address-family commands being entered without the address-family being activated explicitly first, so this problem could only occur if the inheritance was configured some other way, i.e. via the MDA.

Starting config:

```
router bgp 1
  af-group foo address-family ipv4 unicast
  exit
  neighbor 1.2.3.4
    remote-as 1
```

```
      address family ipv4 unicast
        use af-group foo
        policy fred in
      exit
    exit
  exit
```

Change:

```
  router bgp 1
    neighbor 1.2.3.4
      address-family ipv4 unicast
        no use af-group foo
      exit
    exit
  exit
```

This would fail since the **no use** command would be processed before the **address-family** command, leaving the address-family deactivated but still having some config (ie the **policy** command from above).

Restriction:

- Within the HFR system configuration architecture, **Activate** items

  should not be set in any commit where

AFGroup

  items are deleted.

Note: if the original configuration is done using the MDA, the actual problem could be seen using the CLI.

Neighbor groups

IPv4 neighbors can only have IPv4 address families, and same for IPv6. This, combined with 2-stage configuration and batching, causes some un-intuitive behavior with respect to neighbor-groups. For example, the following configuration is valid:

```
  router bgp 1
    address-family ipv6 unicast
    exit
    neighbor-group foo
      address-family ipv4 unicast
      exit
      address-family ipv6 unicast
```

```
    exit
  exit
  neighbor-group bar
    remote-as 1
  exit
  neighbor 1.2.3.4
    use neighbor-group bar
  exit
  neighbor 1:2:3::4
    use neighbor-group bar
  exit
exit
```

However, in this state, no neighbors can use neighbor-group foo, and no address-families can be activated on neighbor-group bar. The following restriction should be observed to avoid the above problem.

Restriction: Neighbor-groups should not be configured with a mixture of IPv4 and IPv6 address families, as they will then not be usable by any neighbors. Note, that it is possible to create such a neighbor-group, within the IOX-SR system configuration architecture, although any attempt to use it will be rejected.

Inheritance and disable

Configuration for neighbors where an inherited item is disabled on the neighbor, and that command has some verification dependencies (e.g. external only config), can lead to situations where removing the neighbor configuration (**no neighbor**) or BGP configuration (**no router bgp**) fails. The following commands are affected:

Neighbor commands

- **dmzlink-bw**
- **ebgp-multihop**
- **local-as**
- **remote-as**
- **ttl-security**

Neighbor address family commands:

- **remove-private-AS**
- **route-reflector-client**
- **send-community-ebgp**
- **send-extended-community-ebgp**

Here is an illustration of the problem, using remote-private-as as an example. The same problem exists with the other commands, following the same general pattern:

```
router bgp 1
  neighbor-group abc
    address-family ipv4 unicast
      remove-private-AS
  nei 10.0.101.1
    remote-as 1
    address-family ipv4 unicast
      remove-private-AS disable
commit

nei 10.0.101.1
  use neighbor-group abc
commit
```

This gives the following config:

```
router bgp 1
 neighbor-group abc
  address-family ipv4 unicast
   remove-private-AS
  !
 !
 neighbor 10.0.101.1
  remote-as 1
  use neighbor-group abc
  address-family ipv4 unicast
   remove-private-AS disable
  !
 !
```

Here **neighbor 10.0.101.1** is internal, but inherits **remove-private-AS** from a neighbor group. Just as the configuration had to be done in two steps, the removal of the neighbor requires two steps.

- remove use of neighbor group
- remove neighbor

Attempting to remove the neighbor in one step gives the following errors:

```
RP/0/0/CPU0:ensoft-gsr6(config)#router bgp 1
RP/0/0/CPU0:ensoft-gsr6(config-bgp)#no neighbor 10.0.101.1
RP/0/0/CPU0:ensoft-gsr6(config-bgp)#commit
RP/0/0/CPU0:Jul 27 17:36:29.349 : config[65703]: %LIBTARCFG-6-COMMIT :
```

```
+Configuration committed by user 'lab'.   Use 'show commit changes 1000002922'
+to view the changes.

% Failed to commit one or more configuration items. Please use 'show
+configuration failed' to view the errors
RP/0/0/CPU0:ensoft-gsr6(config-bgp)#show conf failed
!! CONFIGURATION FAILED DUE TO SEMANTIC ERRORS
router bgp 1
 no neighbor 10.0.101.1
!!% operation failed because some configuration items within the request could
+not be removed. Individual errors are displayed below.
 !
 neighbor 10.0.101.1
  no remote-as
!!% Cannot delete neighbor (10.0.101.1) whilst it has config remaining
  no use neighbor-group
!!% Cannot delete neighbor (10.0.101.1) whilst it has config remaining
  address-family ipv4 unicast
   no remove-private-AS disable
!!% Change would result in internal neighbor (10.0.101.1) with external-only
+config
  !
 !
!
```

Performance and Responsiveness

This section describes commands that are useful when investigating performance or responsiveness issues with BGP. There is also a description of the areas of functionality where some latency of response is expected, because processing is performed by the scanner thread.

Measuring Performance

If poor performance is experienced in a particular repeatable scenario, the following commands can be used to collect information so that the CPU usage of the processes in the system can be determined:

At the start of the test:

- Collect the output of **show version**

- Collect the output of **show process location <node>** for each node where there is a BGP process running.

At the end of the test, for each node where information was collected above:

- Collect the output of **show process location <node>**
- Collect the output of **show process threadname location <node>**

The difference in the CPU time fields between the two sets of output from show process can be used to determine the amount of work performed by each thread in the system.

Note: The amount of CPU time consumed by the idle thread (JID 1 TID 1) gives an indication of how loaded the CPU was during the test. The idle thread runs whenever there is no other thread ready to execute.

BGP processes store statistics about the work done by the process. This information can be displayed using the following command:

- In standalone mode **show bgp all all process performance-statistics detail**
- In distributed mode **show bgp all all process performance-statistics speaker <id>** for the information for a speaker process, and **show bgp all all process performance-statistics brib <id>** for the information for a bRIB process.

The BGP performance statistics can be reset using the following command:

```
clear bgp performance-statistics
```

CPU Busy

If a thread in a BGP process is very busy, it is possible to get an indication of the work that is being performed by that thread using the following commands:

- Output from **show version**
- Output from **show process <jid> location <node>** (JID is the Job ID in the **show process** output).
- Output from **show process threadname <jid> location <node>**
- For each thread that is in Ready state, output from **run attach_process -p <pid> -t <tid> -v -i <n>** (PID and TID appear in the output of **show process <procname>** , **<n>** is the number of times to query the status of the thread). Note this command must be executed on the node on which the process is running.
- Output from **show dll jobid <jid> location <node>**

Areas where latency is expected

Some areas of BGP functionality are processed by the scanner thread. This means that when a change occurs, it will not be processed until the next scanner run. By default, the scanner runs every 60 seconds, though this may be changed using the bgp scan-time configuration command (under the router bgp submode for the generic scanner, and under the address-family submode for the per address family scanner).

The following areas are updated by the scanner thread:

- Aggregate processing
- Default origination
- Conditional advertisement

- Route selection based on invalid nexthop addresses.

BGP memory usage

BGP is the biggest user of memory on the router, and its memory usage is largely dependent on the number of prefixes and paths received from its peers. There are frequent questions on trying to figure out how much of memory BGP is using or issues with BGP running out of memory. One needs to understand the types of memory that comes into play while trying to understand BGP memory usage.

Physical Memory/Systm Memory

This is the amount of RAM available on the router and this memory is shared by all processes running on the router. This memory size can vary from 2 GB to 16 GB depending on the platform and the amount of memory installed. The usage of the physical memory is monitored by wdsysmon and syslog alarms are generated when physical memory usage reaches a threshold, by default 75% (minor alarm) of the total RAM. The severity of the alarm goes up as the threshold increases to 85% (major) and then to 95% (critical). Note, BGP will start to bring down peering once the threshold reaches major level and BGP process would exit once critical threshold is reached. At this point, one needs to make sure that the prefixes/paths received by BGP is within reasonable limit and restart BGP manually. The wdsysmon generated warnings identify the top 5 users of RAM (the following example shows 2 of them):

```
RP/0/RSP0/CPU0:Feb  4 10:58:03.698 PST: wdsysmon[455]:%HA-HA_WD-4-MEMORY_ALARM : Memory threshold crossed: Minor with 613.211MB free
RP/0/RSP0/CPU0:Feb  4 10:58:03.699 PST: wdsysmon[455]:%HA-HA_WD-6-TOP_MEMORY_USERS_INFO : Top 5 consumers of system memory(627528 Kbytes free):
RP/0/RSP0/CPU0:Feb  4 10:58:03.699 PST: wdsysmon[455]:%HA-HA_WD-6-TOP_MEMORY_USER_INFO : 0: Process Name: bgp[1053], pid:581977, Heap usage:
1322980 kbytes, Virtual Shared memory usage: 71572 kbytes.
RP/0/RSP0/CPU0:Feb  4 10:58:03.699 PST: wdsysmon[455]:%HA-HA_WD-6-TOP_MEMORY_USER_INFO : 1: Process Name: ipv4_rib[1146], pid:549192, Heap usage:
651420 kbytes, Virtual Shared memory usage:34484 kbytes.
```

The following commands can be used to check the physical memory (the example shows 6 GB) available on the router.

```
RP/0/RSP1/CPU0:9K_D5#sh memory summary location 0/rsp1/cpu0
Wed Feb  5 15:17:21.027 PST

node:      node0_RSP1_CPU0
------------------------------------------------------------------

Physical Memory: 6144M total (842M available)
Application Memory : 5873M (842M available)
Image: 78M (bootram: 78M)
Reserved: 128M, IOMem: 0, flashfsys: 0
Total shared window: 309M
```

Virtual Memory

Sometimes referred to as rlimit as well and this memory is applicable is per process. The virtual memory limit is dependent on the address space on the router and is 2^32 or 4 GB on the 32-bit QNX system. However, the entire 4 GB is shared by kernel/DLL/shmwin besides the actual process. As a result the effective maximum virtual memory a process can use is considerably less than 4 GB and is dependent on the platform. This limit varies between 1.5 GB on C12K to 1.8 GB on CRS3 to 2.5 GB or ASR9K (after 4.1.0) among the QNX based systems. There is no show command to find out what the maximum usable virtual memory on a platform. The virtual memory used by BGP (one needs to use the jobid of BGP from the - show process bgp - output) at any given moment can be found using the following command.

```
RP/0/RSP1/CPU0:9K_D5#sh memory heap dllname 1053 location 0/rsp1/cpu0
Wed Feb  5 15:16:47.888 PST
  Heapsize 830050304: allocd 826401424, free 2565732, overhead 1083148, high watermark 836734976
```

The heapsize indicates the virtual memory usage by BGP which in this case happens to be about 791 MB (830050304 / (1024 * 1024)). Note that the above output is quite long - only the first couple of lines are shown in the example. The other interesting information here is the high watermark which is the maximum heapsize BGP has used. Besides the steady state memory, BGP uses a lot of transient memory as well for update message generation. So the high watermark can exceed the steady state memory usage by a few hundred megabytes.

Also, to be noted is the fact that standby BGP, if NSR is configured, will use significantly more memory (about 30% additional) compared to the active BGP because of the additional paths that are stored on the standby. So the maximum of the high watermark on the active BGP and the heapsize of the standby BGP will determine the maximum BGP memory usage and this must be less than the rlimit of the platform. Otherwise, BGP will run out of memory which can manifest itself in the form of syslog messages indicating failed malloc/calloc calls.

How does one increase memory

Physical memory on a router can be increased by installing additional RAM provided the maximum memory is not already installed. There is no way to increase the virtual memory on the router. The size is dictated by the address size of the operating system. The address size on QNX is 32 bit and the address size on Linux is 64 bit. Hence the virtual memory limit on Linux is higher than that of QNX.

What is maximum BGP scale

There is no single answer. Maximum BGP scale (prefix and path) that can be achieve depends on the amount of virtual memory available to BGP. The command that is most useful to figure out what is the current BGP scale is the following.

```
RP/0/RSP1/CPU0:9K_D5#sh bgp all all process performance-statistics detail
Mon Feb 10 21:42:58.979 PST

Address Family: IPv4 Labeled-unicast

                    Number        Mem Used
Prefixes:            1297002       133591206
Paths:              12079002       773056128
Path-elems:          1297002        67444104
```

```
Address Family: IPv6 Labeled-unicast

                    Number        Mem Used
Prefixes:             126103        14501845
Paths:               576103        36870592
Path-elems:            126103         6557356

Address Family: IPv4 Unicast

                    Number        Mem Used
Prefixes:            1297002       133591206
Paths:              12079002       773056128
Path-elems:          1297002        67444104

Address Family: IPv6 Unicast

                    Number        Mem Used
Prefixes:             126103        14501845
Paths:               576103        36870592
Path-elems:            126103         6557356
```

The output has been truncated to dispaly the relevant portions only. The key information to note here are the number of prefixes and paths for the different address families. Because Unicast and the Labeled-unicast address families share the same prefix table in the IOS-XR BGP implementation, those prefix and path counts are shown twice in the above example and the Labeled-unicast part can be ignored if the corresponding Unicast part is present. One can see that there are about 1.3M prefixes and about 12M paths in the IPv4 Unicast table. Also there is about 125K prefixes and 575K paths in the IPv6 Unicast table. That's a pretty impressive scale with close to 1.5M prefixes and more than 12M paths. The next question is to figure how much of memory BGP is using with the current scale. That would give us an idea how close we are to the virtual memory limit and whether there is room for growth in scale. Let's look at the BGP memory usage number.

Active BGP

```
RP/0/RSP1/CPU0:9K_D5#sh memory heap dllname 1053 location 0/rsp1/cpu0
Mon Feb 10 21:43:13.161 PST
Malloc summary for pid 577828 process bgp:
  Heapsize 1292886016: allocd 1194801800, free 96647940, overhead 1436276, high watermark 1456205824
```

The active BGP is using a virtual memory of about 1.23 GB (1292886016 / (1024 *1024)) as indicated by the heapsize and the maximum virtual memory used by the active BGP is 1.39 GB (1456205824 / (1024 * 1024)) as indicated by the high watermark.

Standby BGP

```
RP/0/RSP1/CPU0:9K_D5#sh memory heap dllname 1053 location 0/rsp0/cpu0
Mon Feb 10 21:44:13.189 PST
Malloc summary for pid 577828 process bgp:
  Heapsize 1508622336: allocd 1394993048, free 111968748, overhead 1660540, high watermark 1508868096
```

The standby BGP is using a virtual memory of about 1.44 GB (1508622336 / (1024 *1024)) as indicated by the heapsize and the maximum virtual memory used by the standby BGP is the same as indicated by the high watermark since the transient memory needed for update generation is not present on standby. Note that standby BGP will use more memory that the active BGP if NSR is configured.

One needs to look at the high watermark on the active BGP and the heapsize on the standby BGP to make sure whether BGP virtual memory usage is within safe limits of the maximum virtual limit on the platform. As an example, ASR9K allows a maximum virtual memory of 2.5 GB. One needs to make sure that maximum memory used by BGP do not exceed more than 2.3 GB under any circumstances i.e. it is recommended that a 200 MB of safety margin must be maintained.

Detecting BGP memory leaks

Memory used by BGP will increase if the BGP table size grows due to incoming prefixes/paths. The easiest way to determine the source of a memory leak is to get snapshots of memory usage over a period of time. By comparing the difference between the snapshots, the source of the memory leak can more readily be identified provided there has been no increase in the BGP table size. Output from the following commands should be collected 5 times at 30 min intervals.

- Find out the current BGP memory usage - **show memory heap dll <bgp_jobid> location <node>**
- Find out the current BGP scale - **show bgp all all process perf detail location <node>**

There may be cases where the above information is not sufficient to determine whether there is a leak or not and get to the cause of the leak. Additional information may be necessary in such cases

- Output from **show version**
- Output from **run malloc_dump -A -p <jid> -d** before the test. Note the output from this command may be very large, so it is best redirected to a file - i.e. **run malloc_dump -A -p <jid> -d location <loc> > <filename>** and collect the file that is created.
- Collect output from **run malloc_dump -A -p <jid> -d** after the test (wait for 25 minutes as cooldown peroid). By comparing the snapshots, the source of the memory leak can be identified.

BGP or BPM crash

When reporting bgp or bpm crash, please make sure given the following info:

- Output of **show version**

- Output of **show install which component ipv4-bgp**
- output of **show logging**
- Steps to trigger the crash
- Get the decode using workspace from **show install which component ipv4-bgp**
- Do a search from topic.cisco.com to see it is known or not
- If not known, give the pointer to the core file

Typically when SMUs are loaded, please the right set of instruction in the below link to get the complete back-trace (hint: only first or 2nd frame will decode).

http://wwwin-people.cisco.com/mbassin/sbt.html

Note: **if the ws space is nightly or private workspace, Specify that so people can take a look when workspace exists.**

Traffic loss

Figure out the reason for traffic loss

Traffic loss could be caused by so many issues. Consult the graceful restart section to tune the timers first if the traffic loss happens after a BGP process restart or failover when grace-restart is configured.

Information to be collected to analyze the cause of traffic loss

- The topology of the test setup
- The router where the traffic loss happens
- The prefix(es) on which the traffic loss is seen
- The neighbor(s) who advertised those prefixes
- The duration of the traffic loss
- The events which triggers the traffic loss
- Check whether graceful-restart is configured and what are the timer values
- Check the time when BGP entered read-write mode sh bgp <afi> <safi> process perf detail
- Check the time when the first and the last neighbor were established sh bgp vrf <vrf> <afi> <safi> sum
- Check the neighbor from which the prefix was received show bgp vrf <vrf> neighbor <addr>

  to see whether EOR is received or not during read-only mode. If not, most likely you will see traffic loss

- Check the prefix in bgp table such as show bgp vrf <vrf> <afi> <safi> <prefix>

- Check the route in rib such as show route vrf <vrf> <prefix>
- Check the route in cef such as show cef afi or safi <prefix> location <node>
- Collect logs show logging
- Collect BGP traces from both active and standby show bgp trace location <node>
- Collect RIB traces from both active and standby show rib ipv4/ipv6 trace location <node>

  when we instll routes to rib
- Collect CEF traces from both active and standby **show cef ipv4/ipv6 trace location <node>**
  whether route del happens or not during FO.

Expected Traffic loss

Graceful restart is not designed to handle back to back FO (before first FO completes).

If you are doing back to back FO and the second FO happens before the first FO completes (before RIB and FIB

relearn all the routes), you might see traffic loss. This traffic loss is expected.

Graceful restart

BGP graceful restart is enabled on the router using the following config

bgp graceful-restart

Graceful restart behavior is sensitive to a set of configurable timer values whose defaults are meant to handle a few 10s of sessions and a few thousand prefixes. It is necessary to tune these values for any sclae setup.

Tuning the BGP timers

There are four timers that determine the graceful restart behavior. This section describes their roles and their recommended values based on a typical scale setup of 2000 BGP neighbors and 1,000,000 million prefixes on the UUT. These values must be tuned based on the actual topology and the behavior of the neighboring routers.

update-delay

bgp update-delay 600

The BGP update-delay is configured on the UUT. This timer indicates when BGP starts calculating bestpaths after populating the database following a restart. update-delay timer is used in case some peer(s) don’t send End-of-RIB marker or when peers are not GR capable. We see traffic drop if BGP table is not completely re-populated in this update-delay period. The default value of this timer is 2 min. This value needs to be set based on how big a topology we have and how long it takes for all the neighbors to

come up and send their updates. Need to tune it to 10 min or more for a scale network. The correct value can only be deduced through actual observation on the network.

purge-time

```
bgp graceful-restart purge-time 900
```

This timer must be configured both on the UUT as well as on the neighboring routers. This value is programmed in RIB and LSD so that RIB & LSD wait for that long on BGP process restart before they start purging the routes and labels. The purge-time value should be about 50% greater than update-delay time. Thus if BGP takes upto 600 secs to calculate bestpath before it can reclaim the local labels (with LSD) and populate routes and paths back to RIB, the purge-time value should be set at about 900 secs

restart-time

```
bgp graceful-restart restart-time 300
```

The BGP restart-time must be configured both on the UUT as well as on the neighboring routers. This is the time it takes for restarting speaker to re-establish the peering. Default value is 120 secs. Neighboring routers will purge the prefixes from the restarting UUT if the BGP peering is not established withing this time resulting in traffic loss. Recommended value for a scale setup would at least half that of the update-delay period on the UUT.

stalepath-time

```
bgp graceful-restart stalepath-time 1200
```

This timer must be configured on the neighboring routers of the UUT. It is the time it takes for BGP to relearn the routes from the restarting speaker. Default value is 300 secs. Routes remain stale in the forwarding table until they are received from the restarting UUT. This should be set at twice the value of the update-delay of the UUT. Neighboring routers (BGP/RIB/CEF) will purge the routes from the restarting UUT if all routes (and EoR) are not learnt withing this period which will result in traffic drop. This value should be set at 1200 secs if the update-delay on the UUT is 600 secs

Routes are removed immediately after session goes down

Make sure that both sides have advertised and accepted GR. Enabling Graceful Restart on one side does not guarantee the acceptance of the capability for the session. Use show bgp neighbors to find out whether GR has been configured on both sides of the session. In the following example, GR has been enabled on both neighbors.

```
RP/0/0/CPU0:12a1#sh bgp neighbor 10.0.101.1

BGP neighbor is 10.0.101.1, remote AS 1001, local AS 65001, external link
 Remote router ID 10.0.101.1
```

```
    BGP state = Established, up for 00:00:12
    Last read 00:00:11, hold time is 180, keepalive interval is 60 seconds
    Neighbor capabilities:
      Route refresh: advertised
      Address family IPv4 Unicast: advertised and received
    Received 1061 messages, 0 notifications, 0 in queue
    Sent 4310 messages, 0 notifications, 0 in queue
    Minimum time between advertisement runs is 30 seconds

   For Address Family: IPv4 Unicast
    BGP neighbor version 2988
    Update group: 0.1
    Community attribute sent to this neighbor
    Extended community attribute sent to this neighbor
    AF-dependant capabilities:
---> Graceful Restart Capability advertised and received
        Neighbor preserved the forwarding state during latest restart
        Local restart time is 120, RIB purge time is 600 seconds
        Maximum stalepath time is 360 seconds
        Remote Restart time is 120 seconds
    Route refresh request: received 0, sent 0
    Inbound path policy configured
```

Stale route are removed immidiately after session comes back up again

Make sure that neighbor is still advertising GR capability. It is possible that neighbor configuration has been changed and the neighbor is no longer GR capable. If so, all stale routes learned from that neighbor are purged as soon as the session comes back up. Use show bgp neighbors to check capabilities. If the neighbor is still GR capable, then routes are purged if either

- the neighbor does not send any routes and purge timer expires, or
- the neighbor sends an EoR without sending any routes first

BGP Dampening

BGP will damppen the EBGP route if dampening configured for an address family.

Default parameters

BD_DEFAULT_HALFLIFE 15 BD_REUSE_PENALTY 750 BD_SUPPRESS_PENALTY 2000 BD_MAXIMUM_PENALTY 20000 BD_STIME_HALFLIFE_RATIO 4

Is Dampening configured

show running router bgp <as> show bgp afi/safi process

Is the route eBGP route

show bgp <prefix>

What is the current dampening parameter

show bgp <prefix> show bgp process [detail]

BGP NSR

What is NSR

NSR is a self-contained solution to maintain the routing and the forwarding service during:

- RP/DRP fail-over
- Process crash
- In Service Software Upgrade
- Rack OIR in the case of Multi-chassis

No disruption to the routing protocol interaction with other routers

NSR config

Make sure NSR is configured under BGP

RP/0/0/CPU0:R2#show running-config router bgp
router bgp 1
nsr
bgp router-id 3.3.3.3

From 5.3.x onwards, BGP NSR is turned on by default. Hence, you don't need "nsr" configured under router bgp anymore.

NSR requirements

For BGP NSR to start, following requirements should be met:

- At least one established BGP session which is sending prefixes
- All configured VRFs (incl. default-vrf) must have either redistribute enabled or have at least one established BGP session which is sending prefixes.

NSR show commands

show bgp proc performance [standby]

Check the global NSR state on active/standby using this command. **Most Recent**Time stamps next to the state tell us which state we are currently at.

Active BGP NSR State

- ***Standby Ready***: Standby tell active it is ready for nsr to start and active bgp is waiting to get output ready-only mode
- ***TCP Setup***: Active bgp get out of ready-only mode and standby also tell active that it is ready. Active start to do tcp setup such as create session set. Output of **show tcp nsr session brief** will show you the session set created.
- ***TCP Setup Done***: Active bgp finished tcp setup and wait to start tcp init sync once all address family is nsr converged. Output of **show bgp afi/safi summary** will show you whether this address family is converged or not which means all the neighbor has received the updates upto bgp nsr converge version showed up in the above show command. **show bgp afi/safi summary nsr** will show you the AckVer for each neighbor. The AckVer need to be greater than **BGP NSR coverge version**
- ***TCP Init sync***: Active bgp has asked tcp to sync the tcp session from active to standby. Output of **show tcp nsr session brief** will show the session set, number of session in the set, number of session which are UP.
- ***TCP Init Sync done***: Active TCP has done with sync the tcp session from active to standby. In the normal case, all the sessions in the session set should be UP.
- ***FPBSN Processing Done*** : Active bgp is done with FPBSN processing which is used by standby bgp to decide how much data need to to discard when it starts read. Output of **show bgp neighbor x.x.x.x per** will show you the FPBSN value **First Standby PDU Offset**
- ***Update process Done*** : Active bgp is done with update if there is any.
- ***BGP Init Sync***: Active bgp start sync prefix, nbr, orf etc info to standby
- ***BGP Init Sync Done***: Active bgp has sent all the bgp info from active to Standby and wait an Ack from Standby to move to NSR ready
- ***BGP NSR Ready***: Active bgp has received the ACK from standby and move to NSR ready state.

Standby BGP NSR State

- ***Standby read***: Standby bgp is ready to start NSR and has informed active BGP
- ***TCP Replication***: TCP is in the process of replicaiton tcp session for bgp from active to standby such as get fd on standby for a bgp peer session on active.
- ***TCP Init Sync Done***: TCP has done with replicating the tcp session and standby bgp is ready to receive bgp synced infor from active bgp.
- ***BGP NSR Ready***: Standby has received all the bgp info such as prefix, nbr info, counter etc.

show bgp summary standby

Standby version keeps track of what prefixes have been sync'ed to standby, in the steady-state StandbyVer should catch up with **BGP main routing table version**. **InQ** and **OutQ** will show whether there the incoming/outgoing updates pending for procdess

show bgp summary nsr [standby]

This command gives nss summary for a specific afi/sai. We can see the entire state machine transitions for a particular speaker. It also lists the bgp state and

nsr state for a neighbor. tbl ver, sync ver, nbr state and nbr nsr state are things needed to be checked if nsr state machine stuck at TCP setup done state.

```
RP/0/1/CPU0:corvette#show bgp summary nsr

BGP router identifier 1.2.3.4, local AS number 1
BGP generic scan interval 60 secs
Non-stop routing is enabled BGP table state: Active
Table ID: 0xe0000000
BGP main routing table version 11
BGP NSR converge version 11 (Reached)
BGP scan interval 60 secs

BGP is operating in STANDALONE mode.


node0_1_CPU0       Speaker 1

Entered mode  Standby Ready              : Jan  8 06:58:24
Entered mode  TCP NSR Setup              : Jan  8 07:00:28
Entered mode  TCP NSR Setup Done         : Jan  8 07:00:28
Entered mode  TCP Initial Sync           : Jan  8 07:01:31
Entered mode  TCP Initial Sync Done      : Jan  8 07:01:32
Entered mode  FPBSN processing done      : Jan  8 07:01:32
Entered mode  Update processing done     : Jan  8 07:01:32
Entered mode  BGP Initial Sync           : Jan  8 07:01:32
Entered mode  BGP Initial Sync done      : Jan  8 07:01:32
Entered mode  NSR Ready                  : Jan  8 07:01:32


Process  Id   RcvTblVer   bRIB/RIB   LabelVer  ImportVer  SendTblVer  StandbyVer
Speaker  1          11         11         11         11          11          11
bRIB     1          11         11         11         11          11           0

Neighbor        Spk    AS   TblVer  SyncVer   AckVer NBRState     NSRState
10.0.101.1        1     2       11       11       11 Established  NSR Ready
10.0.101.2        1     2       11       11       11 Established  NSR Ready

RP/0/1/CPU0:corvette#show bgp summary nsr standby
```

```
BGP router identifier 1.2.3.4, local AS number 1
BGP generic scan interval 60 secs
Non-stop routing is enabled BGP table state: Active
Table ID: 0xe0000000
BGP main routing table version 11
BGP NSR converge version 11
BGP NSR not converged
BGP scan interval 60 secs

BGP is operating in STANDALONE mode.


node0_0_CPU0       Speaker 1

Entered mode  Standby Ready            : Jan  8 06:58:23
Entered mode  TCP Replication          : Jan  8 07:00:28
Entered mode  TCP Init Sync Done       : Jan  8 07:01:32
Entered mode  NSR Ready                : Jan  8 07:01:32

Process  Id   RcvTblVer   bRIB/RIB   LabelVer  ImportVer  SendTblVer  StandbyVer
Speaker  1        11         -        11        11         11          -
bRIB     1        11        11        11        11         11          0

Neighbor        Spk    AS   TblVer  SyncVer   AckVer NBRState     NSRState
10.0.101.1        1     2      11        0        0 Established  NSR Ready
10.0.101.2        1     2      11        0        0 Established  NSR Ready
```

show bgp <> detail standby

This command lists the path received from active, active bestpath and best path calculated by standby.

```
RP/0/0/CPU0:R2#show bgp 120.0.0.0/24 detail standby
BGP routing table entry for 120.0.0.0/24
Versions:
  Process          bRIB/RIB  SendTblVer
  Speaker               462         462
    Flags:0x360000
Paths: (1 available, best #1)
  Not advertised to any peer
  Path #1: Received by speaker 0
```

```
  Flags:0x1000007
  Local
    20.0.101.1 from 20.0.101.1 (0.0.0.0)
      Origin IGP, localpref 100, valid, internal, best
  Bestpath Received from active speaker 0
  Flags:0x1000007
  Local
    20.0.101.1 from 20.0.101.1 (0.0.0.0)
      Origin IGP, localpref 100, valid, internal
```

show bgp neighbor <> performance-statistics

Use the following command to check the NSR state of a neighbor and some more NSR statistics.

```
RP/0/0/CPU0:R2#show bgp neighbor 20.0.101.1 performance-statistics

BGP neighbor is 20.0.101.1
 Remote AS 1
  Read 1075 messages (34812 bytes) in 1068 calls (time spent: 0.299 secs)
  Read throttled 0 times
  Max socket read size: 1680 bytes
  Actives: 2 , Failed Post Actives 2, Passives 6, Rejected Passives 1
  Processed 35 inbound update messages (time spent: 0.080 secs)
  Wrote 20428 bytes in 1072 calls (time spent: 0.012 secs)
  Processing sub-group: wrote 10 messages in 26 calls (time spent: 0.003 secs)
  Processing write queue: wrote 10 messages in 24 calls (time spent: 0.000 secs)

  Received 1075 messages, 0 notifications, 0 in queue
  Sent 1042 messages, 0 notifications, 0 in queue
  NSR State: NSR Ready
  NSR Statistics:
    First standby offset 1778, First standby PDU offset 2359
    Read 19217 cumulative bytes
    Read 2323 cumulative update bytes
    Wrote 16941 cumulative bytes
    Last Sent SN 16941, Last Ackd SN 16922
```

show bgp table [standby]

We have introduced a new table command which displays all the neighbors associated with a given table (afi/safi). This command gives the ability to find all the PE neighbors and CE neighbors that are configured on the router using a single command as shown in the example below. It also helps in finding the VRF a particular CE neighbors is configured in.

```
RP/0/0/CPU0:corvette#sh bgp table vpnv4 unicast

Thu Mar 19 23:43:41.561 UTC
Neighbor        VRF             Spk    AS   TblVer  InQ OutQ  St/PfxRcd
10.0.101.1      default           0     1     3953    0    0        2
10.0.101.2      default           0     1     3953    0    0        2
10.0.101.3      default           0     1     3953    0    0        1
10.0.101.4      default           0     1     3953    0    0        1
10.0.101.5      default           0     1     3953    0    0        1
10.0.101.6      default           0     1     3953    0    0        1
10.0.101.7      default           0     1     3953    0    0        1
10.0.101.8      default           0     1     3953    0    0        0
10.0.101.9      default           0     1     3953    0    0        0
90.0.0.2        900             0     2       0    0    0 Idle
200.0.0.1       900             0   100     3953    0    0      268
91.0.0.2        901             0     2     3953    0    0        1
201.0.0.1       901             0   100     3953    0    0      268
92.0.0.2        902             0     2     3953    0    0        1
202.0.0.1       902             0   100     3953    0    0      268
93.0.0.2        903             0     2     3953    0    0        1
203.0.0.1       903             0   100     3953    0    0      268
94.0.0.2        904             0     2     3953    0    0        1
204.0.0.1       904             0   100     3953    0    0      268
95.0.0.2        905             0     2     3953    0    0        1
96.0.0.2        906             0     2     3953    0    0        1
97.0.0.2        907             0     2     3953    0    0        1
98.0.0.2        908             0     2     3953    0    0        1
99.0.0.2        909             0     2       0    0    0 Idle
```

The standby version of the same command provide the additional information of how many prefixes have been synced from active to standby. PfxRcd value of 536[536] means that standby has 536 prefixes in its table and [536] implies that all of the 536 were synced from active. A value of 536[0] implies that there are 536 prefixes in the table but none were synced from active. This is an example of a neighbor that came up after global NSR was established at which point both the active as well as the standby reads the updates from the peer independently.

```
RP/0/0/CPU0:corvette#show bgp table vpnv4 unicast standby

Thu Mar 19 23:43:50.396 UTC
Neighbor        VRF             Spk    AS   TblVer  InQ OutQ  St/PfxRcd
10.0.101.1      default           0     1     3953    0    0    2[2]
10.0.101.2      default           0     1     3953    0    0    2[2]
10.0.101.3      default           0     1     3953    0    0    1[1]
10.0.101.4      default           0     1     3953    0    0    1[1]
10.0.101.5      default           0     1     3953    0    0    1[1]
10.0.101.6      default           0     1     3953    0    0    1[1]
10.0.101.7      default           0     1     3953    0    0    1[1]
10.0.101.8      default           0     1     3953    0    0    0[0]
10.0.101.9      default           0     1     3953    0    0    0[0]
90.0.0.2        900             0     2        0    0    0 Idle
200.0.0.1       900             0   100     3953    0    0    536[0]
91.0.0.2        901             0     2     3953    0    0    1[1]
201.0.0.1       901             0   100     3953    0    0    535[536]
92.0.0.2        902             0     2     3953    0    0    1[1]
202.0.0.1       902             0   100     3953    0    0    535[536]
93.0.0.2        903             0     2     3953    0    0    1[1]
203.0.0.1       903             0   100     3953    0    0    268[268]
94.0.0.2        904             0     2     3953    0    0    1[1]
204.0.0.1       904             0   100     3953    0    0    535[536]
95.0.0.2        905             0     2     3953    0    0    1[1]
96.0.0.2        906             0     2     3953    0    0    1[1]
97.0.0.2        907             0     2     3953    0    0    1[1]
98.0.0.2        908             0     2     3953    0    0    1[1]
99.0.0.2        909             0     2        0    0    0 Idle
```

show bgp sessions [standby]

This comand diplays all the BGP neighbors currently configured along with their peering state as well as NSR state. It has filters that can be used to display only those neighbors that are not yet established or those whoich are not NSR ready yet.

```
RP/0/1/CPU0:corvette#sh bgp sessions

Wed Apr 29 18:43:09.608 UTC
```

Neighbor	VRF	Spk	AS	InQ	OutQ	NBRState	NSRState
2.2.2.2	default	0	1	0	0	Idle	None
10.0.101.1	default	0	1	0	0	Idle	None
10.0.101.2	default	0	1	0	0	Idle	None
10.0.101.3	default	0	1	0	0	Idle	None
10.0.101.4	default	0	1	0	0	Idle	None
10.0.101.5	default	0	1	0	0	Idle	None
10.0.101.6	default	0	1	0	0	Idle	None
10.0.101.7	default	0	1	0	0	Idle	None
10.0.101.8	default	0	1	0	0	Idle	None
10.0.101.9	default	0	1	0	0	Idle	None
10.0.101.50	default	0	1	0	0	Idle	None
200.0.0.1	default	0	101	0	0	Established	NSR Ready
201.0.0.1	default	0	101	0	0	Established	NSR Ready
202.0.0.1	default	0	101	0	0	Established	NSR Ready
203.0.0.1	default	0	101	0	0	Established	NSR Ready
204.0.0.1	default	0	101	0	0	Established	NSR Ready
205.0.0.1	default	0	101	0	0	Established	NSR Ready
206.0.0.1	default	0	101	0	0	Established	NSR Ready
207.0.0.1	default	0	101	0	0	Established	NSR Ready
208.0.0.1	default	0	101	0	0	Established	NSR Ready
209.0.0.1	default	0	101	0	0	Established	NSR Ready
210.0.0.1	default	0	101	0	0	Established	NSR Ready
100:200::1	default	0	100	0	0	Idle	None
2020::1002	default	0	1	0	0	Idle	None
90.0.0.2	900	0	2	0	0	Established	NSR Ready
9000::1001	900	0	2	0	0	Established	NSR Ready
91.0.0.2	901	0	2	0	0	Established	NSR Ready
9100::1001	901	0	2	0	0	Established	NSR Ready
92.0.0.2	902	0	2	0	0	Established	NSR Ready
9200::1001	902	0	2	0	0	Established	NSR Ready
93.0.0.2	903	0	2	0	0	Established	NSR Ready
9300::1001	903	0	2	0	0	Established	NSR Ready
94.0.0.2	904	0	2	0	0	Established	NSR Ready
9400::1001	904	0	2	0	0	Established	NSR Ready
95.0.0.2	905	0	2	0	0	Established	NSR Ready
9500::1001	905	0	2	0	0	Established	NSR Ready
96.0.0.2	906	0	2	0	0	Established	NSR Ready
9600::1001	906	0	2	0	0	Established	NSR Ready

```
97.0.0.2       907                0    2    0    0  Established  NSR Ready
9700::1001     907                0    2    0    0  Established  NSR Ready
98.0.0.2       908                0    2    0    0  Established  NSR Ready
9800::1001     908                0    2    0    0  Established  NSR Ready
99.0.0.2       909                0    2    0    0  Established  NSR Ready
9900::1001     909                0    2    0    0  Established  NSR Ready
```

show bgp sessions not-established [standby]

This command show all the bgp neighbor which are not in ESTAB states.

RP/0/1/CPU0:corvette#sh bgp sessions not-established

```
2.2.2.2        default            0    1    0    0  Idle       None
10.0.101.1     default            0    1    0    0  Idle       None
10.0.101.2     default            0    1    0    0  Idle       None
10.0.101.3     default            0    1    0    0  Idle       None
10.0.101.4     default            0    1    0    0  Idle       None
10.0.101.5     default            0    1    0    0  Idle       None
10.0.101.6     default            0    1    0    0  Idle       None
10.0.101.7     default            0    1    0    0  Idle       None
10.0.101.8     default            0    1    0    0  Idle       None
10.0.101.9     default            0    1    0    0  Idle       None
10.0.101.50    default            0    1    0    0  Idle       None
100:200::1     default            0  100    0    0  Idle       None
2020::1002     default            0    1    0    0  Idle       None
```

show bgp sessions not-nsr-ready [standby]

This command show all bgp neighbors which are NOT in NSR ready state on active or standby if there are not any bgp sessions which are not-nsr-ready state . In working case it will give empty output.

RP/0/1/CPU0:corvette#sh bgp sessions not-nsr-ready

Wed Apr 29 18:43:43.163 UTC

```
Neighbor       VRF                Spk   AS   InQ  OutQ  NBRState    NSRState
2.2.2.2        default            0    1    0    0  Idle       None
10.0.101.1     default            0    1    0    0  Idle       None
10.0.101.2     default            0    1    0    0  Idle       None
```

```
10.0.101.3      default          0    1    0    0  Idle       None
10.0.101.4      default          0    1    0    0  Idle       None
10.0.101.5      default          0    1    0    0  Idle       None
10.0.101.6      default          0    1    0    0  Idle       None
10.0.101.7      default          0    1    0    0  Idle       None
10.0.101.8      default          0    1    0    0  Idle       None
10.0.101.9      default          0    1    0    0  Idle       None
10.0.101.50     default          0    1    0    0  Idle       None
100:200::1      default          0  100    0    0  Idle       None
2020::1002      default          0    1    0    0  Idle       None
```

TCP NSR commands

Need location command to check the pcb on active or standby

1. show tcp brief
   check the tcp session state, send/recv buffer, PCB address, port, etc.
2. show tcp detail pcb <pcb address from 1>
   Check the detail tcp pcb info.
3. show tcp nsr client brief [location ...]
   Shows client information, including the application name, instance number, number of session-sets, and the number of sessions and their state.
4. show tcp nsr detail client [<CCB> | all] [location ...]
   More verbose command for client information.
5. show tcp nsr session-set brief
   Check the how many sessions are asked by bgp to replicated and how many are successfully replicated. Compare this with the number in the output of show bgp process perf detail for esablished nbrs.
6. show tcp nsr detail session-set [<SSCB> | all] [location ...]
   Detailed information about the session-set, including time when initial-sync was initiated, etc.
7. show tcp nsr detail pcb <pcb address>
   Give detailed information about NSR state of tcp connection
8. show tcp packet-trace
   Give the detailed for send/rcv packets info for a tcp connection
9. show tcp dump-list x.x.x.x
   Give the list of files for a peer which has flapped/reseted.
10. show tcp dump-file <file name from 6>
    Give the detailed for send/rcv packets info for a reseted tcp connection
11. show tech-support tcp nsr
    Collect tcp nsr tech support info.

TCP NSR statistics command

1. show tcp nsr statistics client [<CCB> | all] [location ...]
   Statistics related to the interaction between the application (client) and the TCP stack.
2. show tcp nsr statistics session-set [<SSCB> | all] [location ...]
   Statistics about the operations on a session-set.
3. show tcp nsr statistics pcb [<PCB> | all] [location ...]
   Statistics pertaining to the send and receive paths for a given session.
4. show tcp nsr statistics summary [location ...]
   An aggregate of all the statistics presented by the above command

Redcon command

show redundancy

Commands used to check what is the nsr state that has been reported to Redcon. It tells whether we are nsr ready or not and if it is not then it gives the reason associated with it. If we are not ready due to BGP then the reason string should be "BGP NSR sessions not sycnchronized".

Commands used to check what is the nsr state that has been reported to Redcon. It tells whether we are nsr ready or not and if it is not then it gives the reason associated with it. If we are not ready due to BGP then the reason string should be "BGP NSR sessions not sycnchronized".

```
RP/0/0/CPU0:R2#show redundancy
Redundancy information for node 0/0/CPU0:
==========================================
Node 0/0/CPU0 is in ACTIVE role
Partner node (0/1/CPU0) is in STANDBY role
Standby node in 0/1/CPU0 is ready
Standby node in 0/1/CPU0 is NSR-ready

Reload and boot info
----------------------
PRP reloaded Mon Nov  5 08:58:46 2007: 3 days, 2 hours, 2 minutes ago
Active node booted Mon Nov  5 08:58:46 2007: 3 days, 2 hours, 2 minutes ago
Standby node boot Mon Nov  5 09:03:03 2007: 3 days, 1 hour, 58 minutes ago
Standby node last went not ready Tue Nov  6 21:57:03 2007: 1 day, 13 hours, 4 minutes ago
Standby node last went ready Tue Nov  6 21:57:08 2007: 1 day, 13 hours, 4 minutes ago
There have been 0 switch-overs since reload
```

****

```
RP/0/0/CPU0:R2#
RP/0/0/CPU0:R2#show redundancy summary
   Active Node    Standby Node
   -----------    ------------
      0/0/CPU0        0/1/CPU0 (Ready, NSR: Ready)
RP/0/0/CPU0:R2#
```

BGP debug commands

debug bgp sync

This gives information about general interaction between active and standby. This debug can come handy while trying to debug issues in the init-sync. We can check the information being synced to standby by active.

debug bgp commlib

This gives details of encoding/decoding between active/standby , brib/spkr etc.

BGP trace commands

```
RP/0/0/CPU0:R2#show bgp trace sync reverse
964 wrapping entries (29696 possible, 10215 filtered, 15197 total)
Nov  7 20:01:05.910 bgp/spkr-sync 0/0/CPU0 t11 Deleting post-it:<Peer closed the session> for 20.0.101.1
Nov  7 20:01:05.910 bgp/spkr-sync 0/0/CPU0 t11 Received post-it:<Peer closed the session> msg:[NSR Steady-sync Event Ack] for 20.0.101.1
Nov  7 20:01:05.910 bgp/spkr-sync 0/0/CPU0 t11 Decoded post-it:<Peer closed the session> msg:[NSR Steady-sync Event Ack] for 20.0.101.1
Nov  7 20:01:05.909 bgp/spkr-sync 0/0/CPU0 t11 Post-it walk completed
Nov  7 20:01:05.909 bgp/spkr-sync 0/0/CPU0 t11 Encoded post-it:<Peer closed the session> msg:[NSR Steady-sync Event Start] for 20.0.101.1
Nov  7 20:01:05.909 bgp/spkr-sync 0/0/CPU0 t11 Send processing of post-it:<Peer closed the session> for 20.0.101.1
Nov  7 20:01:05.908 bgp/spkr-sync 0/0/CPU0 t6  Triggered sync thread for post-it processing
Nov  7 20:01:05.908 bgp/spkr-sync 0/0/CPU0 t6  Created post-it:<Peer closed the session> for 20.0.101.1
Nov  7 20:01:05.908 bgp/spkr-sync 0/0/CPU0 t6  Local event processing of post-it:<Peer closed the session> for 20.0.101.1
Nov  7 20:01:05.908 bgp/spkr-sync 0/0/CPU0 t6  Scheduling processing of post-it:<Peer closed the session> for 20.0.101.1
```

Use the following command to track how BGP NSR state machine moves and whats the current state and the events that caused the transitions in the states

```
RP/0/0/CPU0:R2#show bgp trace sync reverse | inc "active NSR state"
Nov  7 17:38:00.605 bgp/spkr-sync 0/0/CPU0 t11 active NSR state transition from "BGP Initial Sync" to "NSR Ready" -- event rcvd: "BGP Initial sync done"
```

Nov 7 17:38:00.602 bgp/spkr-sync 0/0/CPU0 t11 active NSR state transition from "Update processing done" to "BGP Initial Sync" -- event rcvd: "Standby ready for BGP sync message"
Nov 7 17:38:00.602 bgp/spkr-sync 0/0/CPU0 t11 active NSR state transition from "FPBSN processing done" to "Update processing done" -- event rcvd: "Update Processing Done"
Nov 7 17:38:00.602 bgp/spkr-sync 0/0/CPU0 t11 active NSR state transition from "FPBSN processing done" to "FPBSN processing done" -- event rcvd: "Standby ready for BGP sync message"
Nov 7 17:38:00.602 bgp/spkr-sync 0/0/CPU0 t11 active NSR state transition from "TCP Initial Sync Done" to "FPBSN processing done" -- event rcvd: "FPBSN Processing done"
Nov 7 17:38:00.602 bgp/spkr-sync 0/0/CPU0 t11 active NSR state transition from "TCP Initial Sync" to "TCP Initial Sync Done" -- event rcvd: "TCP initial sync done"
Nov 7 17:37:59.598 bgp/spkr-sync 0/0/CPU0 t11 active NSR state transition from "TCP NSR Setup Done" to "TCP Initial Sync" -- event rcvd: "End of Convergence"
Nov 7 17:36:06.575 bgp/spkr-sync 0/0/CPU0 t11 active NSR state transition from "TCP NSR Setup" to "TCP NSR Setup Done" -- event rcvd: "TCP NSR setup done"
Nov 7 17:36:06.559 bgp/spkr-sync 0/0/CPU0 t11 active NSR state transition from "Standby Ready" to "TCP NSR Setup" -- event rcvd: "End of read-only"
Nov 7 17:35:52.877 bgp/spkr-sync 0/0/CPU0 t11 active NSR state transition from "None" to "Standby Ready" -- event rcvd: "Standby ready message"

BGP NSR triage examples

BGP global NSR state is ready - REDCON says not Ready

- **show redundancy to see wether standby node is ready or not, nsr not ready is caused by bgp or not**
- **show bgp process perf detail [standby] to see whether established sessions are same or not on standby and active**
- **show tcp nsr session-set brief [location xx] to see how many sessions are replicated on active and standby.**
- **show bgp sessions not-established [standby] to see sessions which are not ESTABLISHED (are they supposed to be up?)**
- **show bgp sessions not-nsr-ready [standby] to see sessions which are not NSR ready**
- **show bgp [vrf <vrf>] afi/safi neighbor x.x.x.x to see why nbr are not up**
- **show logging | inc x.x.x.x to see the nbr up/down timestamp and reason**
- **show bgp trace location all | inc x.x.x.x to see nbr related bgp trace (what's going on with this neighbor).**
- **show tcp brief location xxx | inc x.x.x.x to check tcp pcb for this nbr (session state, send/recv buffer)**
- **show tcp detail <pcb address from above> location to check detail tcp pcb info for this nbr**
- **show tcp packtrace <pcb address> most recent in/out packets for this nbr**
- **show tcp dump-file x.x.x.x location <> most recent in/out packets for reseted pcb for this nbr**

- **If the issue is still unknown, get bgp team involved with your find and collect much info follow the guideline.**

BGP NSR not-Ready

- **show redundancy to see wether standby node is ready or not**
- **show running router bgp to see whether nsr is configured or not.**
- **Confirm if both active/standby BGP processes are running and all threads are created.**
- **Confirm if both active/standby BGP processes have detected each other (from 5.0.0 onwards). Show bgp nsr [standby] should show partner endpoint with valid data.**
- **show bgp process perf on active and standby to check the global nsr state to see which state it is in**
- **For case where active bgp is in TCP setup done, it means bgp is waiting for address family convergence. BGP will not start NSR unless all the neighbor has received the updates upto BGP nsr converge version mentioned before**
- **For case where active bgp is in TCP Init-Syne, it means BGP is either waiting for at least one session to come up or waiting on TCP to complete TCP sync. Use "show bgp sessions" to confirm that there are established sessions to be synced.**
- **show bgp all all converge will show you all the address family configured for bgp**
- **For each address family, get the output of show bgp <afi> <safi> summary nsr to see whether this address family is NSR converged or not.**
- **If one address family is not converged, try to figure out which neighbor is not catching up from show bgp <afi> <afi> summary nsr. Looking for neighbour whose Ackver and or TblVer is less than BGP nsr converge version**
- **For vpnv4 or vpnv6 address family show bgp vrf all <afi> <safi> summary nsr also need to be checked.**
- **If a neighbor is identified, please check the InQ or OutQ for this neighobr in show bgp <afi> <safi> summary nsr to see whether there are incoming update or outgoing update pending or not.**
- **Check show tcp brief for that neighbor to wether they are data in the send/receive buffer or not.**
- **Check to see whether the neighbor is in any update group show bgp update-group neighbor x.x.xx**
- **Check configuration to see whether there is anything wrong for this neighbor.**
- **Report the issue to bgp team with the above show output, what your finding is so far and follow the guideline**

Why does it take so long for BGP to get NSR ready?

BGP starts the NSR related setup only after all the address-families have converged or reached read-write state. This period is determined by how long it takes for all the neighbors to come up and send updates. BGP starts out in read-only mode when it doesn't generate any update even after receiving updates from neighbors. BGP exits the read-only mode either after it has received EoR from all the neighbors or the update-delay timer has expired (whichever happens first). Subsequenly, it calculates bestpaths, allocates labels, updates the RIB and enters normal or read-write mode. At this point it generates updates and continues to do so whenever there are changes in the BGP table. The NSR synchronization between active and standby starts only after all the BGP address-families have converged i.e. BGP has entered read-write mode for all the address-families. This makes sure that initial BGP convergence is not impacted due to

NSR related synchronization between active and standby.

The way we do this is roughly the following:

1) BGP comes up, reads config, and initializes data structures,

2) Establish all sessions with peers,

3) For each AF,

a) Enter read-only mode: receive updates from peers & install them in BGP table.

b) When EoR is received from all peers, walk the table and calculate best path for each prefix. This process essentially advances the BGP table version from 0 (when the AF was in read-only mode) to roughly the #of prefixes in BGP table (in your example, it would be 200001).

c) We cache this value as what we call as "NSR convergence version", i.e. we wait for this table state to be sent to all peers (& get acknowledged) before we start init-sync with standby.

d) Once the best paths are calculated, BGP would install all these prefixes in RIB and start generating updates to peers. Eventually, it will send all prefixes to all peers and get acknowledgement (in terms of TCP ACK) from them. We have an algorithmic way of mapping the TCP ACK sequence number to a BGP table version number -- that's how we come to know that all peers have received the BGP table state upto a certain version#.

e) When this happens, we would declare "BGP NSR converged" and move the NSR state machine to start init-sync with standby (More specifically, we would wait for this state (BGP NSR converged) to happen for all configured AFs before we move the NSR state machine).

4) From then on, any time prefixes change (flap / more paths arrive/get withdrawn etc), the BGP table version keeps getting advanced --- this is what would be reflected in the "BGP main routing table version". As you may also know, the version is a monotonically increasing number that dentifies the BGP table state at any moment of time.

BGP Scoped sync

Scoped sync is a mechanism used by active BGP to selectively sync one of more neighbors to the standby after BGP NSR readiness has been reached. This is an extremely light-weight mechanism where only the prefixes received from the affected neighbors are synced to standby. This is different from steady-state sync of a neighbor that gets established after BGP becomes NSR ready. Active BGP syncs the entire BGP table (bestpaths only) to the standby while doing a steady-state sync for every neighbor that comes up after BGP has become NSR ready.

When does BGP do scoped sync

Scoped sync is performed on a neighbor that is in established state but NSR can not be maintained on the standby because of either resource limitation or error condition. The following events will lead to scoped sync at present

1. Oper down

There is a special situation when TCP may discontinue the NSR support for a neighbor when there are too many retransmissions for a neighbor. In this situation, TCP sends what is called an OPER_DOWN notification to both active and standby BGP indicating that active TCP will no longer try to sync the standby TCP for this neighbor and instead active TCP will try the traditional datapath for this neighbor. On receiving the OPER_DOWN notification, the standby BGP resets the neighbor and the active BGP moves the NSR state of the neighbor to OPER_DOWN indicating that the neighbor is no longer NSR capable. Active BGP also tries scoped sync to see whether NSR can be reestablished for this neighbor after 180 secs when the TCP retransmission issue may have

```
gone away.
RP/0/1/CPU0:Jun  5 04:46:31.222 : tcp[397]: %IP-TCP_NSR-3-FAILURE : 206.0.0.2:57098 <-> 206.0.0.1:179:: NSR terminated because of Retransmission threshold exceeded
RP/0/1/CPU0:Jun  5 04:46:31.226 : bgp[134]: %ROUTING-BGP-3-NBR_NSR_DISABLED : NSR disabled on nbr 206.0.0.1 due to TCP retransmissions
RP/0/0/CPU0:Jun  5 04:46:31.228 : bgp[134]: %ROUTING-BGP-5-STANDBY : neighbor 206.0.0.1 Down on Standby - Peer closing down the session (VRF: red)
RP/0/1/CPU0:Jun  5 04:46:31.238 : bgp[134]: %ROUTING-BGP-5-NSR_STATE_CHANGE : Changed state to Not NSR-Ready
RP/0/1/CPU0:Jun  5 04:46:31.239 : tcp[397]: %IP-TCP_NSR-3-FAILURE : 208.0.0.2:38703 <-> 208.0.0.1:179:: NSR terminated because of Retransmission threshold exceeded
RP/0/0/CPU0:Jun  5 04:46:31.244 : bgp[134]: %ROUTING-BGP-5-STANDBY : neighbor 208.0.0.1 Down on Standby - Peer closing down the session (VRF: red)
RP/0/1/CPU0:Jun  5 04:46:31.549 : tcp[397]: %IP-TCP_NSR-3-FAILURE : 209.0.0.2:30964 <-> 209.0.0.1:179:: NSR terminated because of Retransmission threshold exceeded
RP/0/0/CPU0:Jun  5 04:46:31.554 : bgp[134]: %ROUTING-BGP-5-STANDBY : neighbor 209.0.0.1 Down on Standby - Peer closing down the session (VRF: red)
RP/0/1/CPU0:Jun  5 04:46:32.098 : tcp[397]: %IP-TCP_NSR-3-FAILURE : 207.0.0.2:32419 <-> 207.0.0.1:179:: NSR terminated because of Retransmission threshold exceeded
RP/0/0/CPU0:Jun  5 04:46:32.103 : bgp[134]: %ROUTING-BGP-5-STANDBY : neighbor 207.0.0.1 Down on Standby - Peer closing down the session (VRF: red)
RP/0/1/CPU0:Jun  5 04:46:32.340 : tcp[397]: %IP-TCP-3-BADAUTH : Invalid MD5 digest from 207.0.0.1:179 to 207.0.0.2:32419
RP/0/1/CPU0:Jun  5 04:46:32.589 : tcp[397]: %IP-TCP_NSR-3-FAILURE : 210.0.0.2:52036 <-> 210.0.0.1:179:: NSR terminated because of Retransmission threshold exceeded
RP/0/0/CPU0:Jun  5 04:46:32.595 : bgp[134]: %ROUTING-BGP-5-STANDBY : neighbor 210.0.0.1 Down on Standby - Peer closing down the session (VRF: red)

RP/0/1/CPU0:corvette#
RP/0/1/CPU0:corvette#sh bgp vrf red sum nsr

Fri Jun  5 04:46:55.382 UTC
BGP VRF red, state: Active
BGP Route Distinguisher: 1:1
VRF ID: 0x6000000c
BGP router identifier 2.3.4.5, local AS number 1
Non-stop routing is enabled
BGP table state: Active
Table ID: 0xe000000c
BGP main routing table version 11615
BGP NSR Initial initsync version 11573 (Reached)

BGP is operating in STANDALONE mode.


node0_1_CPU0       Speaker
```

```
Entered mode  Standby Ready              : Jun  5 00:27:17
Entered mode  TCP NSR Setup              : Jun  5 00:32:20
Entered mode  TCP NSR Setup Done         : Jun  5 00:32:20
Entered mode  TCP Initial Sync           : Jun  5 00:32:31
Entered mode  TCP Initial Sync Done      : Jun  5 00:32:32
Entered mode  FPBSN processing done      : Jun  5 00:32:32
Entered mode  Update processing done     : Jun  5 00:32:32
Entered mode  BGP Initial Sync           : Jun  5 00:32:32
Entered mode  BGP Initial Sync done      : Jun  5 00:32:33
Entered mode  NSR Ready                  : Jun  5 00:32:33

Current BGP NSR state - NSR Ready achieved at: Jun  5 00:32:33
NSR State NOT READY notified to Redcon at: Jun  5 04:46:31

Process      RcvTblVer   bRIB/RIB   LabelVer  ImportVer  SendTblVer  StandbyVer
Speaker        11615      11615      11615      11615      11615       11615

Neighbor       Spk    AS   TblVer  SyncVer   AckVer NBRState     NSRState
206.0.0.1       0   101    11615    11615    11615 Established  TCP Oper Down
207.0.0.1       0   101    11615    11615    11615 Established  TCP Oper Down
208.0.0.1       0   101    11615    11615    11615 Established  TCP Oper Down
209.0.0.1       0   101    11615    11615    11615 Established  TCP Oper Down
210.0.0.1       0   101    11615    11615    11615 Established  TCP Oper Down

RP/0/1/CPU0:corvette#sh bgp vrf red sum nsr standby

Fri Jun  5 04:47:03.510 UTC
BGP VRF red, state: Active
BGP Route Distinguisher: 1:1
VRF ID: 0x6000000c
BGP router identifier 2.3.4.5, local AS number 1
Non-stop routing is enabled
BGP table state: Active
Table ID: 0xe000000c
BGP main routing table version 11615
BGP NSR Initial initsync version 0 (Not Reached)

BGP is operating in STANDALONE mode.
```

```
node0_0_CPU0       Speaker

Entered mode  Standby Ready            : Jun  5 00:27:17
Entered mode  TCP Replication          : Jun  5 00:32:20
Entered mode  TCP Init Sync Done       : Jun  5 00:32:32
Entered mode  NSR Ready                : Jun  5 00:32:33

Process      RcvTblVer   bRIB/RIB   LabelVer  ImportVer  SendTblVer  StandbyVer
Speaker        11615        0         0         0          0          0

Neighbor       Spk    AS   TblVer  SyncVer   AckVer NBRState     NSRState
206.0.0.1       0   101       0       0       1 Idle        None
207.0.0.1       0   101       0       0       1 Idle        None
208.0.0.1       0   101       0       0       1 Idle        None
209.0.0.1       0   101       0       0       1 Idle        None
210.0.0.1       0   101       0       0       1 Idle        None

RP/0/1/CPU0:corvette#
```

2. Header sync error on standby

On rare occasions, there has been cases where a neighbor on standby goes down with header syncronization error. This is caused by packet accounting mismatch between BGP and TCP. The root cause of this is still under investigation. However, BGP uses the scoped stnc mechanism to recover from this error. The standby BGP resets the neighbor and sends a message to active BGP to initiate scoped sync operation on this neighbor. The active BGP changes the NSR state of this neighbor to OPER_DOWN, waits for 60 secs and then syncs the neighbor to standby. Note, that the ios_msg from BGP would indicate a standby neighbor reset.

```
RP/0/1/CPU0:Jun  5 04:46:31.226 : bgp[134]: %ROUTING-BGP-3-NBR_NSR_DISABLED : NSR disabled on nbr 206.0.0.1 due to Standby nbr reset
```

3. Strict check failure

The essence of NSR is replicate the active BGP on the standby so that there should be difference seen by the external peers when the failover happens and the standby BGP takes as active. One of the challenges of this replication effort is to maintain absolute sanity between the BGP tables on the active and those on the standby. BGP tries its utmost to make sure that the tables are identical at two different times. One is after init sync when BGP on standby performs a strict check to make sure that for every neighbor on the standby, the number of received prefixes from active matches with what active promised (this is the accepted prefix count from the neighbor) at the beginning of the sync. The other occasion when BGP performs this prefix check is after a scoped sync. At this occassion, the scrict check is performed first by the active BGP which tries to macth the number of prefixes it synced to the standby for a given neighbor

matches with the number fo prefixes it received from the neighbor. The standby also performs a strict check to make sure that it has received the expected number for prefixes for a neighbor after active notifies standby that scoped sycn is done. Depending on who detects the mismatch, a scoped sycn is initiated by either active or standby BGP for a neighbor for which the mismatch was found. Note that the scoped sync is attempted only once in case the mismartch persists and an ios_msg would notify the user to investigate further. This is not expected to happen.

Future usage of scoped sync

Scoped sync provides a lightweight, per neighbor sync mechanism, which can be used in a number of circumstances in the future. First example that comes to mind is to substitute the current BGP steady state sync where a neighbor comes up after global NSR has been reached with a scoped sync. There are other possible usage which may have more profound effect on the usability of NSR as a technology.

When does BGP restart the NSR state Machine

There are cases where BGP NSR state machine will be restarted because of failures which can not be recovered from by doing selective sync of neighbor. The following is a list of such cases.

- QAD timeout
- TCP Init Sync fails for 5 times in a row
- TCP session set creation fails
- TCP restart
- Inconsistency between active and standby regarding NSR state msg
- TCP OPER_DOWN is received before BGP finishes init-sync
- Repeat TCP OPER_DOWN for a neighbor is received while the previous one is being processed

QAD timeout

When we see QAD timeout messages, we should check the state of QAD using (a) CPU usage at both RPs and (b) QAD ping's average round-trip time.

```
RP/0/RP0/CPU0:CRS-IP-BR#run nsr_qad_ping -c 100
NSR Ping from node0_RP0_CPU0 to node0_RP1_CPU0 on LOW queue
Sending 100, 100-byte pings, timeout is 2000 msec
!!!!!!!!!!!!!!!!!!.!!!!!!!!!!!!!!!!!!
```

The new behavior from 4.0 is to not have nbr session changes (c and d) affect nsr readiness of the router. We do not want to keep the entire router from being nsr ready if few nbrs are not nsr ready after global nsr sync.

Will BGP session flap with NSR enabled when restart BGP process

When bgp process on active node get restarted, the remote side will see bgp session flap. It has nothing to do with NSR and/or GR feature is enabled or not.

If "nsr process-failure switchover" is configured and if active bgp process CRASHED, then a switchover will be triggered and the remote side will not see bgp session flap.

INIT-SYNC failure on shutting interface

When an interface is shut, session(s) will eventually flap because of hold-timer expiry. Before that happens, TCP will retransmit several times. When the retransmissions cross a certain threshold, TCP first brings down NSR. So, BGP tries to perform NSR init-sync once again before the session flaps. NSR init-sync will be successful only if there is connectivity to the peer. So, it is expected to see the following error messages.

RP/0/5/CPU0:Sep 23 11:44:07.270 : bgp[128]: %ROUTING-BGP-5-IO_INITSYNC_FAILED : 'ip-tcp' detected the 'try again' condition 'Initial sync operation timed out' - set id 1
RP/0/0/CPU0:Sep 23 11:44:07.300 : bgp[128]: %ROUTING-BGP-5-IO_INITSYNC_FAILED : 'ip-tcp' detected the 'try again' condition 'Initial sync operation timed out' - set id 1
RP/0/5/CPU0:Sep 23 11:45:49.602 : bgp[128]: %ROUTING-BGP-5-ADJCHANGE : neighbor 50.1.4.1 Down - BGP Notification sent, hold time expired (VRF: default)

Explanation for %ROUTING-BGP-3-INITSYNC_PFX_MISMATCH

On the active BGP, the router thread is responsible for processing incoming updates. However, we stop processing new updates form the input queue when we start NSR syncing a neighbor in steady state. We sync a neighbor in steady state under two different situations -

1) a new neighbor is configured after BGP has become NSR ready 2) a neighbor has gone down and has come back up 3) a neighbor has received an OPER_DOWN and we sync it after 3 min

The sync thread is responsible for syncing the neighbor related information and prefixes/paths to the standby. We can have a situation where the router thread picks up an update from the input queue and starts processing. It gets swapped out before it has a chance to process the entire update. In the mean time, the sync thread gets scheduled and starts syncing the standby. Two kinds of information are synced by the sync thread

1) neighbor counters first 2) prefixes/paths/attributes received from the neighbor second

It is possible that after sending the neighbors counters which includes the number of prefixes received from the neighbor, the sync thread gets swapped out and router thread gets scheduled.

The router thread continue to process the unfinished update which causes new prefixes to be added or deleted to the BGP table.

Next, when the sync thread starts the prefix/path/attribute sync, the BGP table may have changed causing the initial prefix counter that was sent to be out of sync with the number of prefixes in the table and which are synced to the standby.

The standby performs an accounting of the number of prefixes expected (based on the neighbor counter sent) with the actual number of prefixes received and throws up this ios-msg if there is a mismatch. In such a case, BGP will try to sync the neighbor one more time.

Difference in NSR Implementation between IOS and IOS-XR

- Config in XR is under global router bgp submode – which enables NSR for all the neighbors.
  In IOS the NSR config is enabled per neighbor since they don't support NSR for all the PE-CE neighbors.
- In XR the replication of the incoming updates for Active-RP and S-RP happens in hardware via fabric multicast.
  In IOS the replication happens in software. - It is first received by Active TCP, sent over to S-TCP. The data is acked to the peer and made available to BGP only after S-TCP acks the receipt of data. Due to software replication it affects how much scale can be supported.
- In the send direction, similar approach is used. A-TCP first sends the tx data to S-TCP, waits for an ack and then puts the data out on the wire. In IOS-XR S-RP TCP saves the entire update message so that it can resend the PDU if there is a FO and the peer doesn't ack the segment. In IOS There was one optimization that they attempted to do: which is to just sync the size of the segment, instead of the complete data. Upon FO, the idea was to generate dummy data to fit that size, in case the peer asks for retransmission. This approach has several gotchas and it is my understanding that it was later discarded in favor of syncing complete data.
- In XR the NSR is supported for iBGP/eBGP peering across all AFIs.
  In IOS NSR is not supported for iBGP peers (PE-PE or PE-RR peering) but GR is recommanded. This means due to GR there are still limitations of running into blackhole of traffic if multiple network events happen at the same time.
- In XR we support NSR for up to 2400 BGP sessions – I don't see IOS scaling up to that level.
- Some IOS NSR docs can be found under http://wwwin-eng.cisco.com/Eng/OIBU/WWW/SSO/IOS_Documents
  IOS-XR BGP NSR Docs: EDCS-642709 BGP NSR TOI

When does BGP restart the NSR state machine?

There are cases where BGP NSR state machine will be restarted because of failures, which can not be recovered from by doing selective sync of neighbor. The following is a list of such cases.

- QAD/NPL timeout
- TCP Init Sync fails for 5 times in a row
- TCP session set creation fails
- TCP restart
- Inconsistency between active and standby regarding NSR state msg
- TCP OPER_DOWN is received before BGP finishes init-sync
- Repeat TCP OPER_DOWN for a neighbor is received while the previous one is being processed

BGP NSR Related Changes in Recent Releases

BGP NSR Changes from 4.3.0 onwards:

- BGP doesn't restart NSR if we hit TCP OPER-DOWN while BGP Global init-sync is going on. The affected session is put in scoped-sync list and BGP will attempt scoped-sync for that session.
- If a BGP neighbor undergoes TCP OPER-DOWN while scoped-sync was going on, BGP will no longer attempt scoped-sync for that session. That neighbor will be marked TCP Oper-Down permanently until it's removed and configured again.

BGP NSR Changes from 5.0.0 onwards

- **Network Packet Layer (NPL): The communication between the RPs is now moving from QAD to NPL infrastructure. NPL is a platform-agnostic infra that all the applications is required to use from 5.0.0 onwards. On Classic XR (CRS, ASR9K, GSR) platforms, NPL will rid over QAD. However, QAD is not present in Panini. BGP has moved from QAD library to NPL library now. All the references to QAD in show outputs, traces, IOS Msgs will be replaced with NPL.**
  Refer to wikicentral.cisco.com/display/PROJECT/NG-XR+NSR+Client+Packet+Library+%28aka+NSR+QAD+lib+in+XR%29 for more information.
- **Detection Using Directory Services (DS): BGP process on A-RP and S-RP now uses DS to detect it's counter-part on the partner Node/RP. DS provides the infrastructure for processes to detect other processes and have a Liveliness mechanism. BGP takes advantage of this to avoid unnecessary msgs while and to react quickly when the partner node goes down. The information on partner process detection is present in show bgp nsr [standby] ouput. Sample output snippet is given below. The Local Endpoint section is about the local process's DS End-point. The partner endpoints detected are next. If the partner is detected, the end-point information will have value else it'll be blank (--). E.g. The Standby process is detected below, however secondary active BGP process is not yet detected.**

```
RP/0/RP0/CPU0:pir-issu-5#show bgp nsr
. . .
Local Endpoint:
  ISSU/HA Role: Primary/Active
  Node ID: 0x1000
  NPL Handle: 0x24a80e0
  Endpoint len: 42
  Bytestream:
    0A0B6267 702D6465 6661756C 74108020
    18032080 B0CFC0DC ACD2B380 0128EC97
    0330DF8D 0138AC81 C008
```

```
Entity:Standby (2):
  Sync-group: Sync-grp Entity:Standby Sync-group (1)
  ISSU/HA Role: Primary/Standby
  Node ID: 0x1100, Wheel ID: 1
  Max Msg Size: 9216, Exp Seq No: 1196
  NPL Handle: 0x7f09a40a3260
  Endpoint len: 42
  Bytestream:
    0A0B6267 702D6465 6661756C 74108022
    18032080 8ABFB089 E786F780 0128CCE9
    0330E5CF 0238AC81 C408

Entity:Secondary Active (4):
  Sync-group: Sync-grp Entity:ISSU Sync-group (3)
  ISSU/HA Role: --/--
  Node ID: 0x0, Wheel ID: 2
  Max Msg Size: 0, Exp Seq No: 0
  NPL Handle: 0x0
  Endpoint len: 0
  Bytestream:
    None
```

The scope of BGP DS detection is limited within a given BGP instance. That is, BGP process will not detect other BGP processes in other instance. The way this is done is using the BGP instance name in the end-point publication and registration.

When BGP is not NSR Ready or if there are some issues with A-BGP and S-BGP communication. It’s a good idea to first check if A-BGP and S-BGP have detected each other. BGP will not discard the packets received from the partner BGP process that’s not detected yet. To find out whether BGP is dropping any incoming msgs from an undetected process or whether BGP is dropping any outcoming msgs to an undetected process, pl refer to the NPL Statistics section of show bgp nsr output. The Message Send Drops and Message Recv Drops below indicate if any msg from/to an undetected process is dropped.

```
NPL Statistics:

  Messages Sent       : 0          ACKs Received       : 0
  Messages Received   : 0          ACKs Sent           : 0
  Send Failures       : 0          Send ACK Failures   : 0
  Suspends            : 0          Resumes             : 0
```

```
 Messages Processed  : 0            Out of sequence drops: 0
 Messages Send Drops : 0            Messages Recv Drops : 0
```

- NPL Packet statistics: NPL packet statistics is now displayed in show bgp nsr output. This information is very useful to find out packet rx/tx/drop information for a given application. Sample output is shown below.

```
RP/0/RP0/CPU0:pir-issu-5#show bgp nsr
. . .

NSR NPL Packet Statistics
Prio Queue: Low
----------------
      Msg-type                                Number
      ------------------------------------------------------------
      Sent Data                :                 7

      Recv Data                :                1196


      ****Drop Stats****

      Msg-type            Drop-reason                 Number
      ------------------------------------------------------------
      Send Drop:     <None>
      Recv Drop:     <None>


Prio Queue: High
----------------
```

BGP NSR Changes from 5.2.0 onwards

- BGP Hot Standby: Backup BGP processes viz. Prim-Stdby, Secondary Actv and Stdby BGP processes operate in fully hot standby mode when NSR/ISSU is going on. What this means is that BGP on these nodes will perform all the operations just like the Primary Actv BGP. This includes, label allocation, route import, RIB download, etc. Except import, other operations were omitted on backup BGP earlier. However, from 5.2.0 all the servers (RIB, LSD) are hot standby and hence backup BGP is hot standby too. Note: If BGP NSR is not enabled, BGP will not be hot standby and hence these operations will not occur on backup BGP process.

- BGP Go-Active Serialization: BGP Go-Active operation is orchestrated where various threads perform go-active operation in a serial fashion. The new change from 5.2.0 is that the Route Download operation is extended from just RIB go-active to RT-DNLD go-active where the routes are downloaded to all the concerned servers (RIB for IPv4/v6 VRF, PIM for MDT, MVPN, L2VPN-Mgr for VPLS/VPWS/EVPN/MSPW, etc). During R-DNLD go-active, BGP sync-thread will trigger RT-DNLD to scanner, rib and l2vpn threads to download routes for respective AFI. Trigger is sent per AFI that's enabled. Once the RT-DNLD is done, pulse is sent back to sync-thread. Once all the RT-DNLD-DONE pulses are received, sync-thread continues with Go-Active operation. If BGP is not NSR Ready after RPFO, check if any RT-DNLD is still pending. Use "show bgp all all process performance detail to check if Route Download is pending for any AFI or not. Look for "Route Download Go-Active-Done notif pending" under a given AFI. If you see it, the RT-DNLD-DONE is not received yet for that AFI.

- BGP NSR Restart Changes: BGP will no longer restart NSR when a large number (>75) sessions undergo TCP OPER-DOWN. BGP will remain NSR Ready and the affected sessions will be synced via scoped-sync mechanism. It is operators responsibility to make sure the BGP sessions are NSR Ready before issuing Redundancy Switchover (RPFO) as the session(s) that were not NSR Ready before RPFO will flap and can potentially cause network churn/traffic misrouting and/or drop.

BGP NSR Changes from 5.2.2 onwards

BGP NSR Restart Changes: When Active BGP hits TCP Oper-down for all the ESTABLISHED peers, there are no more sessions that are protected. Hence, if RPFO is issued at that time, all the BGP sessions will flap. Hence, it was decided to bring BGP NSR down when all the established BGP peers are in TCP Oper-down state. This will prevent RPFO to be issued until BGP again becomes NSR Ready.

Why is BGP Not NSR Ready after RPFO?

There can be multiple reasons why BGP is not NSR Ready after RPFO. Triage this issue in the order below.

- Check if Standby RP (S-RP) has booted up and is in Ready State. Use show redundancy. The S-RP state should be Ready. If it's "Not Ready", that means S-RP is still booting or the booting has failed. In that case, BGP NSR won't initiate. Check with platform team to debug it further. Else continue to next step.
- Check BGP process state on S-RP. Use "show process bgp location <S-RP-LOC>. It should be in Running state. If it's in Hold/Restarting/Not Yet Spawned state, involve placed/sysmgr team to help. Else continue to next step.
- From 5.0.0 onwards, BGP uses DS to detect the counter-part on partner node. See if A-BGP and S-BGP have detected each other. Use "show bgp nsr" and "show bgp nsr standby" to see if detection is done. Pl visit the "Detection Using Directory Services" section more information. If DS detection is done correctly, continue to next step.
- Check what stage BGP NSR is stuck in. Use "show bgp nsr" and check for "Entered mode" for a list of all the modes/stages BGP NSR has gone through. Look for the reason for not moving forward. See "BGP NSR state not ready:" reason right after the "Entered mode" output.

In the snippet below, BGP NSR is stuck in TCP Initial Sync stage. It's waiting on TCP initial sync to finish before proceeding to next stage.

========== Post Failover Summary for Active instance ==========

```
Node              Process          Read     Write    Inbound
node0_RP0_CPU0      Speaker          11.04      17.65       0.01
  Entered mode  Standby Ready              : Aug 17 23:02:45
  Entered mode  TCP NSR Setup              : Aug 17 23:02:45
  Entered mode  TCP NSR Setup Done          : Aug 17 23:02:45
  Entered mode  TCP Initial Sync            : Aug 17 23:02:45
Current BGP NSR state - TCP Initial Sync achieved at: Aug 17 23:02:45
BGP NSR state not ready: TCP Initsync in progress
NSR State NOT READY notified to Rmf at: Aug 17 23:01:36
Waiting for sessions to be added to NSR session set
```

- **If you see that BGP is not NSR Ready to waiting on TCP or due to TCP init-sync timeout, pl consult TCP team (xr-transports-dev@cisco.com). Otherwise proceed to next step.**
- **Please gather following outputs and provide to BGP Team (q-bgp-dev) to triage the issue further.**
  - **Show bgp nsr**
  - **Show bgp nsr standby**
  - **Show tech routing bgp**

Why does one need to configure a /32 route to an ASBR peer

Perform this task to configure a static route to an Inter-AS or CSC-CE peer.

When one configures an Inter-AS or CSC peer, BGP allocates a label for a /32 route to that peer and performs a implicit-NULL label rewrite. When forwarding a labeled packet to the peer, CEF removes the top label from the label stack; however, in such an instance, CEF expects a /32 route to the peer. This task ensures that there is, in fact, a /32 route to the peer. Please note that...

o A /32 route to the neighbor is not required to establish BGP peering. A route using a shorter prefix length will also work.
o A shorter prefix length route is not associated with the allocated label; even though the BGP session comes up between the peers, without the static route, forwarding will not work.
o Unlike in IOS, this static configuration is not done automatically by IOS-XR BGP due to the clear separation in the process boundaries between BGP and static and each process must take care of it's own configuration.

Does IOS-XR Handle slow-peer detection

In XR, BGP produces UPDATE messages for the update-group using the update-gen thread and they get queued up.
The io_write thread picks up the messages from the queue and writes them to individual TCP socket buffer for

each neighbor. The threads update-gen and io_write work in parallel. So if one neighbor is slow for a short period of time, that wouldn't affect other neighbors from receiving the UPDATE messages. However, until an UPDATE message has been sent to all neighbors in the update-group, that message wouldn't be deleted. So the message queues may start to build up. We have configurable limits on how much memory can be used for storing outstanding UPDATE messages. Only when we hit the memory limit would we stop generating UPDATEs.

So the bottom line is, if a neighboring router becomes slow/stuck temporarily (for a "small" period of time), no one else gets affected. This is the usual case in all XR deployments. In such cases, in our experience, the overhead of trying to change update-groups and later remerging them is more costly and not worth it.

What is IOS-XR BGP write limit

BGP write-limit is used to provide an upper bound on the number of transient BGP update messages queued in the system.

For the purpose of update generation, neighbors are part of an unique update-group on an per-address-family basis. The update-groups can contain one or more subgroups. A neighbor can be part of only one subgroup at a given time. BGP updates are generated on a sub-group basis. This is a 2-stage process:

(1) Allocate, format and queue update messages. There's a single common queue of messages per subgrp that is shared by all neighbors in the subgroup. The allocated message can be one of the following sizes (in bytes): 80, 110, 210, 310, 600, 1100, 4300
(2) Write the queued messages to the neighbor socket. The message is freed after all the neighbors in the subgroup have written it to their respective sockets.

BGP write-limit configuration allows the user to specify the maximum number of update messages that can be formatted or queued under a subgroup. Two different values can be specified:

(a) Per-update-group limit: This is the maximum number of update messages that are formatted during one update generation run on a subgroup. The next run for this subgroup will be scheduled only if there's atleast one neighbor that has written the previously formatted messages in the subgroup message queue.
(b) Global limit: This is the maximum number of updates messages that can be queued, across all the update-groups (and their subgroups) in the bgp process.

Considerations

(1) The default values for write-limit (50K, 250K) are set to work for the most common deployment cases. These values can be tuned for a specific deployment scenario.

(2) The factors to consider when tuning are:
  (a) Transient memory: Higher values result in higher transient memory consumption
  (b) Convergence time: Higher values can decrease convergence time while lower values can increase convergence time in a scaled network during events like session flap, route refresh, process restart, etc.
(3) Note that write-limit is specified as message count values. If a large number of larger size messages are formatted during update generation, there will be a corresponding increase in transient memory consumption.
(4) If a subgroup contains a mix of fast and slow neighbors, that can result in more update generations runs (and hence more update messages queued) on that subgroup.
(5) The update-group write-limit is applied separately for each address-family that is negotiated on the neighbor session. However the global limit is common across all update-groups across all address-families.
(6) If the bgp process contains a mix of fast and slow update-groups, the slow update-groups can slow down the fast update-groups if/when the global write-limit is reached. This should be taken into consideration when tuning the global limit.

Multiinstance BGP

Currently, it is possible to configure upto 4 BGP instances on a single RP. All BGP instances can be configured with either the same AS or unique AS. Only the following address-families can be shared between two or more instances with the constraint that the VRF name and the RD must be unique across instances

- o VPNv4 Unicast
- o VPNv6 Unicast
- o VPNv4 Multicast
- o VPNv6 Multicast
- o IPv4 RT-filter
- o IPv4 MDT
- o IPv4 MVPN
- o IPv6 MVPN
- o VPNv4 Flowspec
- o VPNv6 Flowspec

Multithreaded BGP architecture

IOS-XR BGP has been designed as a multi-threaded application from ground up to take advantage of the SMP architecture of the IOS-XR platforms - both physical as well as virtual. IOS-XR employs a fair preemptive scheduler where each thread is allocated a fixed time-slice (4 msec) so that no thread can monopolize the system. One of the bigger challenges in a multi-threaded application is the mutex contention while accessing a common resource. BGP threads have been designed with a very specific set of well-defined tasks to minimize this contention. As a result, an operation like receiving and processing a prefix update/withdraw consists of a set of serial operations where the threads are invoked in sequence (io-read -> router -> import -> label -> rib -> update-gen -> sync -> event) to minimize contention.

Although, not much of parallelism can be achieved while processing a single prefix update/withdraw, except in the cases where a thread is blocked over a mutex or a synchronous API call and the next thread get scheduled, the scenario changes significantly while processing a large number of prefixes. This is especially true during the

process bring-up where the router is learning and processing a large number of prefixes or during a major interface event like an interface flap. BGP convergence during such event can be significantly improved in the presence of SMP where multiple BGP threads can possibly get scheduled across the different CPUs at the same time or they can get CPU time more frequently compared to the case of a platform with a single CPU. There are a few other factors that impacts the overall BGP convergence.

How does multiple CPUs help BGP convergence?

(a) BGP convergence improves as the number of available CPU increases. It is likely that this improvement will not be perceptible beyond 8 since it is unlikely that more than 8 BGP threads can run in parallel even when all the stars align

(b) BGP prefixes are stored in tries in separate tables. A separate table is created based on AFI/SAFI pair or VRF. It should be noted that there are likely to be more mutex contention among the various threads if they are working on the same trie in a given table. The chance of contention is much less if the threads are working on different tables. So BGP convergence improvement is more noticeable on a multi-CPU platform if multiple BGP tables are being updated at the same time. The improvement is less if only a single table is being impacted.

How many BGP threads are there?

BGP creates about 18 threads based on what has been configured. However, there can be additional threads associated with the BGP process which are not directly controlled by BGP. BGP interacts with various servers by linking with the DLLs provides by the servers. Many of these DLLs BGP links with launches threads of their own. As a result, the total number of BGP threads in a run-time environment can be much higher (up to 25 in QNX and 30 on Linux).

What do the different BGP threads do?

IO Control Thread

IO control thread is the main BGP thread. The IO control thread mainly maintains the TCP session management including creation of listen sockets, maintaining per neighbor connection timers, processing active as well as passive connections, managing connection collision and installing the LPTS static policies. IO control thread also performs the TCP session replication during NSR setup.

IO Read Thread

IO read thread is the dedicated thread to handle read from TCP for all neighbors in a non-blocking manner. Responsible for reading data from TCP, Convert the received data to BGP messages and handle partial data. It queues the update message into each neighbor's InQ for router thread to do the processing. It also helps in NSR setup.

Router Thread

Router thread is primarily responsible for processing the messages received from neighbors. The responsibilities can be broadly categorized as

(a) Remove the update messages from the neighbor's readQ and process it (b) After parsing update messages received from peers, validate the prefixes, attributes as well we nexthops (c) Insert the attributes into the attribute cache and routes into the BGP per address-family BGP routing table (d) Calculate the bestpaths after inserting new path (e) Delete the withdrawn paths (f) Initiate resetting of neighbor(s) and if needed generate notifications

Router thread refers to the nexthop trie to validate/store the information about whether the nexthop is valid, local and reachable. Nexthop validation is done at the time when BGP receives the updates.

Label Thread

Label thread is responsible for label management of L3VPN routes. BGP requests LSD for local label (in-label) allocation. XR supports three label modes (a) per prefix (b) per vrf (c) per CE (resilient per-CE)

BGP sends label request to the LSD by passing in a suitable "context" data structure depending on the label model; the context may be the table-id, the prefix and the next-hop-set id or their combination as may be the case. It also uses something called mpls rewrite; mpls output information (moi) and forwarding path information (fpi) in the label request. BGP assigns labels for vpn routes for the following cases:

(a) routes learnt from a CE neighbor (b) routes learnt from the RIB via redistribution (c) ASBR scenarios, option B and C cases. (d) CsC (e) Next-hop-self configured on the RR/ASBR (f) 6PE (g) Relevant label policy is configured for the address family

BGP has timers to retain labels for a specific duration in case of LSD restart so as not to cause traffic outage while LSD comes back up.

Import Thread

A bgp speaker retains VPN routes only if it has a L2/L3 VRF which is configured with an import RT that matches an export RT on the received route. Once the router thread has accepted a VPN route the import thread does the actual import operation. To do this BGP builds a list of tables which are configured with the said extended community; a copy of the net and path is made and placed inside the relevant BGP VRF table (except in the same RD case in which case we just mark as imported). Usually only the best path of a given (RD, prefix) is imported. However with multi-path configured, more than one eligible paths may be imported. BGP also employs optional batching via a configurable import timer called bgp import-delay to optimize import.

In some cases as in an option B ASBR, BGP is configured with a "retain route-target all" to accept every received VPN route, regardless of presence of VRFs or configured import RT extcom. This is independent of the import operation. Sometimes import can be forced due to the result of a config change, for instance an import RT change, in which case BGP can do a scoped walk (RD or VRF specific) or else an entire walk over the BGP table for that address-family.

RIB Thread(s)

There can be up to two different RIB threads to interact with IPV4 and IPv6 RIB processes. One RIB thread binds to IPv4 RIB process and is responsible for updating IPv4 RIB with bestpaths from IPv4 unicast and Multicast tables. The second RIB thread will bind to the IPv6 RIB process and is responsible for updating the IPv6 RIB with bestpaths from the IPv6 address-family tables. Both the threads share the same code and only a context data contains the information about each RIB thread.

The RIB thread is also responsible for receiving the redistributed routes from RIB and are populated into the BGP tables. RIB thread uses the client library provided by the RIB process to interact with RIB. A table-policy can be configured in BGP per global IPv4/IPv6 address family which can influence (including drop) the routes being installed in the RIB. RIB threads also can download attributes in RIB for NetFlow application.

Upgen Thread

This thread is responsible for walking the prefix trie for a given table and generating update/withdraw messages and copying them into the messages queues in the update-groups.

IO Write Thread

This threads writes BGP messages generated by the upd-gen thread to TCP sockets in a non-blocking manner. It also writes messages generated by io-control thread such as open, notification etc.

Critical Event Thread

This thread is responsible for responding to the critical events that impact BGP bestpaths. These include

(a) Nexthop notification from RIB (b) BFD notifications (c) Fast External Failover notifications

On receiving these notification, BGP recalculates the bestpath and sends out withdraws/update as required.

Event Thread

The event (also called scanner) thread performs the following periodic tasks.

(a) Perform garbage collection in gateway cache tables. (b) Scan the nexthop cache and generate list of invalid nexthops. (c) Invalidate gateway cache. (d) Flush attribute

cache (e) Flush attribute-info cache associated with aspath, community-list, extended community-list, rrinfo etc. (g) Flush neighbor info cache

Besides the generic scanner, a per-AF scanner is run which performs following AF dependent tasks. (a) install sourced networks, validate the redistributed and sourced networks. (b) calculate aggregates. (c) check and make a list of conditional advertisement changes (d) remove the nets with no paths from the table after making sure that withdraws have been sent (e) if valid paths are present check the gateway metric associated with all the paths and calculate the new bestpath if necessary. (f) check the sourced routes and validate them

Apart from these periodic tasks the scanner also supports some of the BGP event and tasks such as (a) multipath change (b) route-refresh (c) soft-reconfig (d) version warp (e) update-group list merge (f) finishing reset of neighbors (g) processing link bandwidth changes etc.

Sync Thread

This thread dedicated to handle the syncing of BGP database between active and standby RPs for NSR feature. On active node, the thread is called sync-active thread and on standby the thread is called sync-standby thread. The active-sync thread is responsible for syncing BGP data such as neighbor, prefix, path, attributes etc. and events such as NSR state driven events, neighbor events such as reset, policy change etc to standby. The sync-standby thread is responsible for receiving those data and updating it's database. After FO, the sync-standby thread becomes the sync-active thread.

Mgmt Thread

The mgmt thread is responsible for processing the CLI/XML/SNMP requests. It also provides the necessary information in response to a show command request.

Keepalive thread

BGP need to handle keepalive messages for thousands of neighbor some of which could have very aggressive timers. This separate thread has been created to make sure that BGP can service keepalive for all the neighbors within the respective keepalive periods. It writes the pre-formatted keepalive message to TCP.

External Thread

This thread is dedicated to handle the interaction with the RPKI server and BMP server. It maintains separate TCP connections to the respective servers.

L2VPN Thread

This thread interacts with the L2VPN Manager eVPN and MSPW address-families. It processes bridge update/delete, bridge RT add/delete and NLRI update/delete events from the L2VPN Manager, and is used to install remote EVPN and MSPW NLRIs to L2VPN Manager.

MIB Trap Thread

This thread services the neighbor state change relates MIB trap generation and delivery.

ATTACHMENTS

transports_triage.ppt

Comments

lthangli

08-18-2016 05:29 PM

Hi Santosh,

Unable to access the link that you mentioned.. Could you help out here?

http://wwwin-eng.cisco.com/Eng/OIBU/WWW/IOX_IO_Infrastructure/Triage%20Training/transports_triage.pp...

Thanks,

Lal.

santsha2

08-18-2016 05:42 PM

Hi Lal,

I have attached the PPT to this doc. You can dowload it from here.

Thanks,
Santosh

lthangli

08-18-2016 05:50 PM

Thanks Santhosh. I found it.

/Lal

muadnan

04-18-2018 01:13 PM

Thanks also from where to get QNX Virtual Memory allocation for later releases?

Powered by Khoros